

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-100863-126709

**Aplikácia umelej inteligencie pri plánovaní
pohybu robotických systémov**

Bakalárska práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-100863-126709

Aplikácia umelej inteligencie pri plánovaní pohybu robotických systémov

Bakalárska práca

Študijný program:	robotika a kybernetika
Študijný odbor:	kybernetika
Školiace pracovisko:	Ústav robotiky a kybernetiky
Školiteľ:	Ing. Jakub Ivan



ZADANIE BAKALÁRSKEJ PRÁCE

Študent: **Matvii Boltian**
ID študenta: 126709
Študijný program: robotika a kybernetika
Študijný odbor: kybernetika
Vedúci práce: Ing. Jakub Ivan
Vedúci pracoviska: prof. Ing. František Duchoň, PhD.
Miesto vypracovania: Ústav robotiky a kybernetiky

Názov práce: **Aplikácia umelej inteligencie pri plánovaní pohybu robotických systémov**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Plánovanie pohybu robotov je fundamentálnou výzvou robotiky, kde tradičné algoritmy môžu zlyhávať v komplexných prostrediach. Umelá inteligencia prináša nové paradigmy pre riešenie týchto problémov. Táto bakalárska práca sa zameriava na teoretickú analýzu využitia umelej inteligencie špecificky pre úlohy plánovania pohybu robotov. Preskúma princípy, súčasné prístupy a zhodnotí ich potenciál a limity pre generovanie bezpečných a efektívnych trajektórií v rôznych robotických aplikáciách.

Ciele:

1. Analyzujte a vysvetlite, ako sa princípy neurónových sietí (NN), najmä posilňovaného učenia, aplikujú na riešenie problémov plánovania pohybu robotov.
2. Vykonajte systematickú rešerš a kategorizáciu súčasných prístupov využívajúcich NN pre rôzne aspekty plánovania pohybu.
3. Porovnajte rôzne AI stratégie pre plánovanie pohybu, zhodnoťte kvalitu generovaných trajektórií a škálovateľnosť pre roboty s mnohými stupňami voľnosti.
4. Posúďte, či sú tieto metódy vhodné na reálne nasadenie (časová náročnosť, robustnosť, adaptabilita).
5. Zosumarizujte kľúčové poznatky práce.

Termín odovzdania bakalárskej práce: 05. 06. 2026
Dátum schválenia zadania bakalárskej práce: 13. 10. 2025
Zadanie bakalárskej práce schválil: doc. Ing. Eva Miklovičová, PhD. – garantka študijného programu

Podakovanie

Osobitná vďaka patrí mojej rodine – mame, otcovi a bratovi, ktorí mi stáli po boku aj v najťažších chvíľach a poskytovali mi neustálu podporu.

Rád by som sa podakoval aj všetkým vyučujúcim nášho inštitútu, ktorí nás počas bakalárskeho štúdia sprevádzali svojimi vedomosťami, ochotou a iniciatívnym prístupom. Moja vďaka patrí aj priateľom, ktorých som spoznal počas štúdia na tomto krásnom mieste, ako aj všetkým ľuďom, ktorí mi boli nablízku a podporovali ma.

Abstrakt

Táto bakalárska práca je venovaná analýze metód umelej inteligencie v úlohách plánovania pohybu robotov. Práca sa zameriava na porovnanie klasických plánovacích algoritmov s novšími prístupmi založenými na neurónových sieťach a ich hlbšiu analýzu, pričom osobitná pozornosť je venovaná plánovaniu pohybu manipulátora vo vopred definovaných statických scénach.

Ťažisko práce tvorí učenie s posilňovaním a jeho použiteľnosť pri generovaní trajektórií pre robotické rameno Franka Emika Panda so siedmimi stupňami voľnosti. Teoretická časť systematizuje základné prístupy k plánovaniu pohybu a k neurónovým sieťam, zatiaľ čo praktická časť experimentálne porovnáva vybrané optimalizačné, vzorkovacie a RL algoritmy podľa kritérií kvality trajektórie, úspešnosti riešenia a časového správania. Práca zároveň hodnotí robustnosť, adaptabilitu a limity týchto metód z pohľadu ich praktického nasadenia. Výsledkom práce je syntéza hlavných výhod a nevýhod jednotlivých prístupov pri plánovaní pohybu robotického manipulátora.

Kľúčové slová

plánovanie pohybu, umelá inteligencia, neurónové siete, posilňovacie učenie, konfiguračný priestor, robotika.

Abstract

This bachelor's thesis is devoted to the analysis of artificial intelligence methods in robot motion planning tasks. The thesis focuses on comparing classical planning algorithms with newer approaches based on neural networks and their in-depth analysis, with particular attention paid to planning manipulator motion in predefined static scenes.

The core of the work is reinforcement learning and its applicability in generating trajectories for the Franka Emika Panda robotic arm with seven degrees of freedom. The theoretical section systematizes basic approaches to motion planning and neural networks, while the practical section experimentally compares selected optimization, sampling, and RL algorithms based on trajectory quality, solution success, and time behavior. The thesis also evaluates the robustness, adaptability, and limitations of these methods from the perspective of their practical deployment. The result of the thesis is a synthesis of the main advantages and disadvantages of individual approaches to motion planning for a robotic manipulator.

Keywords

motion planning, artificial intelligence, neural networks, reinforcement learning, configuration space, robotics.

Obsah

Úvod	12
1 Robotické systémy	13
1.1 Klasifikácia a typy	13
1.1.1 Definícia a klasifikácia robotov	13
1.2 Teoretické aspekty pohybu robotov	13
1.2.1 Kinematika robota	13
1.2.2 Dynamika robota	15
1.3 Výzvy a trendy vývoja	16
2 Plánovanie pohybu robotov	17
2.1 Hlavné úlohy plánovania pohybu	17
2.1.1 Plánovanie trasy a plánovanie trajektórie	17
2.1.2 Konfiguračný priestor	17
2.1.3 Hlavné ciele štúdia	18
2.2 Klasické metódy plánovania pohybu	19
2.2.1 Vyhľadávanie v grafoch	19
2.2.2 Optimalizačné metódy	20
2.2.3 Metódy potenciálneho poľa	20
2.2.4 Metódy vzorkovania	21
3 Neurónové siete a ich využitie v plánovaní pohybu	24
3.1 Základy neurónových sietí	24
3.1.1 Biologická analógia a matematický model neurónu	24
3.1.2 Aktivačné funkcie v neurónových sieťach	26
3.1.3 Schopnosť generalizácie	27
3.2 Architektúra neurónových sietí	28
3.2.1 Viacvrstvové neurónové siete	28
3.2.2 Konvolučné neurónové siete	29
3.2.3 Rekurentné neurónové siete	30
3.3 Učenie s posilňovaním	31
3.3.1 Základné princípy RL	33
3.3.2 Algoritmy RL	33
3.3.3 Uplatnenie RL v praktických úlohách	34
4 Experimenty	35
4.1 Hardvér a softvér	35

4.2	Kritériá hodnotenia	36
4.3	Implementácia	38
4.3.1	Definícia robota a úlohy	38
4.3.2	Trénovanie a testovanie RL algoritmov	39
4.4	Hodnotenie porovnávacích experimentov	39
4.4.1	Experiment č. 1: voľný priestor	40
4.4.2	Experiment č. 2: malá stena	44
4.4.3	Experiment č. 3: veľká závesná stena	48
4.5	Hodnotenie vzajomných RL experimentov	52
4.5.1	Správanie RL v experimente č. 1	52
4.5.2	Správanie RL v experimente č. 2	53
4.5.3	Správanie RL v experimente č. 3	54
4.6	Výsledky experimentov	55
Záver		59
Literatúra		60
	Použitie nástrojov umelej inteligencie	64
A Definícia úlohy		65
B Nastavenia príslušných algoritmov		66

Zoznam obrázkov a tabuliek

Obrázok 1	Príklad priamej kinematickej úlohy	15
Obrázok 2	C_{space} , C_{free} a C_{obs} pre artikulovaného robota s dvoma kĺbmi [6].	18
Obrázok 3	Ilustrácia potenciálneho poľa pre plánovanie pohybu [16] . .	21
Obrázok 4	Vzrastajúci v čase RRT [19].	22
Obrázok 5	Štruktúra biologického neurónu [22]	24
Obrázok 6	Matematický model neurónu	25
Obrázok 7	Viacvrstvová neurónová sieť (MLP)	28
Obrázok 8	CNN na rozpoznávanie napísaných čísel [24]	29
Obrázok 9	Schéma rekurentnej neurónovej siete	31
Obrázok 10	Schéma učenia sa s posilňovaním	32
Obrázok 11	Klasifikácia algoritmov RL	32
Obrázok 12	Schéma experimentu	35
Obrázok 13	Simulované prostredie pre experimenty	38
Obrázok 14	Experiment 1: Porovnanie výsledkov všetkých algoritmov . .	40
Obrázok 15	Experiment 1: Priebeh parametrov v čase pri algoritme CHOMP	40
Obrázok 16	Experiment 1: Priebeh parametrov v čase pri algoritme PRM	41
Obrázok 17	Experiment 1: Priebeh parametrov v čase pri algoritme PPO	41
Obrázok 18	Experiment 1: Priebeh parametrov v čase pri algoritme SAC	41
Obrázok 19	Experiment 1: Priebeh parametrov v čase pri algoritme TD3	42
Obrázok 20	Experiment 1: Priebeh parametrov v čase pri algoritme TQC	42
Obrázok 21	Experiment 2: Porovnanie výsledkov všetkých algoritmov . .	44
Obrázok 22	Experiment 2: Priebeh parametrov v čase pri algoritme STOMP	44
Obrázok 23	Experiment 2: Priebeh parametrov v čase pri algoritme PRM	45
Obrázok 24	Experiment 2: Priebeh parametrov v čase pri algoritme PPO	45
Obrázok 25	Experiment 2: Priebeh parametrov v čase pri algoritme SAC	45
Obrázok 26	Experiment 2: Priebeh parametrov v čase pri algoritme TD3	46
Obrázok 27	Experiment 2: Priebeh parametrov v čase pri algoritme TQC	46
Obrázok 28	Experiment 3: Porovnanie výsledkov všetkých algoritmov . .	48
Obrázok 29	Experiment 3: Priebeh parametrov v čase pri algoritme TrajOpt	48
Obrázok 30	Experiment 3: Priebeh parametrov v čase pri algoritme RRT-Connect	49
Obrázok 31	Experiment 3: Priebeh parametrov v čase pri algoritme PPO	49
Obrázok 32	Experiment 3: Priebeh parametrov v čase pri algoritme SAC	49

Obrázok 33	Experiment 3: Priebeh parametrov v čase pri algoritme TD3	50
Obrázok 34	Experiment 3: Priebeh parametrov v čase pri algoritme TQC	50
Obrázok 35	Rozptyl trajektórie v rámci tej istej politiky, experiment 1	52
Obrázok 36	Rozptyl trajektórie v rámci tej istej politiky, experiment 2	53
Obrázok 37	Rozptyl trajektórie v rámci tej istej politiky, experiment 3	54
Tabuľka 1	Porovnanie algoritmov pre experiment 1	43
Tabuľka 2	Porovnanie algoritmov pre experiment 2	47
Tabuľka 3	Porovnanie algoritmov pre experiment 3	51
Tabuľka 4	Porovnanie metrík RL-algortimov pre experiment 1	53
Tabuľka 5	Porovnanie metrík RL-algortimov pre experiment 2	54
Tabuľka 6	Porovnanie metrík RL-algortimov pre experiment 3	55

Zoznam značiek a skratiek

ISO - Medzinárodná organizácia pre normalizáciu z angl. *International Organization for Standardization*

IEEE - Inštitút inžinierov elektrotechniky a elektroniky z angl. *Institute of Electrical and Electronics Engineers*

DOF - stupne voľnosti z angl. *Degrees of Freedom*

UAV - bezpilotné lietadlo z angl. *Unmanned Aerial Vehicle*

UUV - bezpilotný podvodný vozidlo z angl. *Unmanned Underwater Vehicle*

AI - umelá inteligencia z angl. *Artificial Intelligence*

LLM - veľký jazykový model z angl. *Large Language Model*

RL - učenie s posilňovaním z angl. *Reinforcement Learning*

MPC - modelovo prediktívne riadenie z angl. *Model Predictive Control*

C-space - konfiguračný priestor z angl. *Configuration Space*

RRT - rýchly náhodný strom z angl. *Rapidly-exploring Random Tree*

PRM - pravdepodobnostný roadmap z angl. *Probabilistic Roadmap*

TrajOpt - optimalizácia trajektórie z angl. *Trajectory Optimization*

STOMP - stochastická optimalizácia trajektórie z angl. *Stochastic Trajectory Optimization for Motion Planning*

CHOMP - kontinuálna optimalizácia trajektórie z angl. *Covariant Hamiltonian Optimization for Motion Planning*

DNN - hlboká neurónová sieť z angl. *Deep Neural Network*

NN - neurónová sieť z angl. *Neural Network*

TLU - prahový logický člen z angl. *Threshold Logic Unit*

ReLU - lineárna jednotka s rektifikáciou z angl. *Rectified Linear Unit*

MLP - viacvrstvový perceptrón z angl. *Multilayer Perceptron*

CNN - konvolučná neurónová sieť z angl. *Convolutional Neural Network*

RNN - rekurentná neurónová sieť z angl. *Recurrent Neural Network*

DQN - hlboká Q-sieť z angl. *Deep Q-Network*

BPTT - spätná propagácia v čase z angl. *Backpropagation Through Time*

CSV - hodnoty oddelené čiarkou z angl. *Comma-Separated Values*

MSE - stredná kvadratická chyba z angl. *Mean Squared Error*

BCE - binárna krížová entropia z angl. *Binary Cross-Entropy*

SQP - sekvenčné kvadratické programovanie z angl. *Sequential Quadratic Programming*

SOTA - aktuálny stav techniky z angl. *State of the Art*

PPO - Proximal Policy Optimization z angl. *Proximal Policy Optimization*

SAC - Soft Actor-Critic z angl. *Soft Actor-Critic*

TD3 - oneskorený dvojité deterministický policy gradient z angl. *Twin Delayed Deep Deterministic Policy Gradient*

TQC - Truncated Quantile Critic z angl. *Truncated Quantile Critic*

OOP - objektovo orientované programovanie z angl. *Object-Oriented Programming*

EE - koncový efektor z angl. *End Effector*

URDF - univerzálny formát popisu robotov z angl. *Universal Robot Description Format*

ROS - robotický operačný systém z angl. *Robot Operating System*

Úvod

Plánovanie pohybu patrí medzi základné úlohy robotiky. Jeho cieľom je nájsť takú postupnosť akcií alebo trajektóriu, ktorá robotu umožní presun z počiatočného stavu do cieľového stavu pri rešpektovaní kolíznych, kinematických a často aj dynamických obmedzení. V praxi sa pritom sledujú aj ďalšie kritériá, napríklad čas vykonania, energetická náročnosť alebo plynulosť pohybu.

S rozvojom robotických aplikácií sa nároky na plánovanie pohybu zvyšujú. Klasické metódy, ako sú grafové algoritmy, vzorkovacie prístupy alebo optimalizačné metódy, poskytujú pri dobre opísanom prostredí spoľahlivé riešenia, ich použitie však môže byť pri vysokej dimenzii konfiguračného priestoru výpočtovo náročné. Táto otázka je obzvlášť dôležitá pri manipulátoroch s viacerými stupňami voľnosti, kde rastie počet možných konfigurácií aj náročnosť vyhýbania sa prekážkam.

V posledných rokoch preto rastie záujem aj o prístupy založené na neurónových sieťach a o učenie s posilňovaním, ktoré umožňujú aproximovať rozhodovaciu politiku priamo zo skúsenosti. Tieto metódy sľubujú rýchlu inferenciu po natrénovaní modelu a schopnosť pracovať aj vo vysokorozmerných stavových priestoroch, na druhej strane však prinášajú otázky interpretovateľnosti, robustnosti a prenositeľnosti medzi prostrediami.

Cieľom tejto práce je teoreticky opísať vybrané klasické a neurónové prístupy k plánovaniu pohybu a následne ich experimentálne porovnať na manipulátore Franka Emika Panda v troch statických scénach s rozdielnou geometriou prekážok. Hodnotenie sa sústreďuje na kvalitu trajektórií, úspešnosť riešenia, časové správanie a praktickú použiteľnosť jednotlivých prístupov. Osobitná pozornosť sa venuje tomu, do akej miery môžu algoritmy RL predstavovať konkurencieschopnú alternatívu ku klasickým plánovačom v úlohách plánovania pohybu manipulátora.

1 Robotické systémy

Táto časť je venovaná definícii základných princípov a vlastností robotických systémov. Obsahuje rozbor podľa rôznych kritérií, kinematiku a dynamiku, ako aj základné úlohy a trendy v tejto oblasti.

1.1 Klasifikácia a typy

Táto časť stručne uvádza základné východiská pre definovanie a delenie robotických systémov.

1.1.1 Definícia a klasifikácia robotov

Robot možno chápať ako fyzický systém určený na vykonávanie úloh v reálnom prostredí prostredníctvom senzorov, akčných členov a riadenia [1, 2]. V tomto zmysle nejde len o mechanický manipulátor, ale o systém, ktorý prepája vnímanie, rozhodovanie a vykonanie pohybu. Robotické systémy možno zároveň deliť podľa oblasti použitia, mobility, úrovne autonómie a kinematickej štruktúry [1]. Z hľadiska použitia sa rozlišujú najmä priemyselné, servisné a výskumné roboty. Z hľadiska mobility ide o stacionárne alebo mobilné systémy, pričom mobilné roboty môžu pôsobiť na zemi, vo vzduchu, pod vodou alebo vo vesmíre. Dôležitým parametrom je aj počet stupňov voľnosti, ktorý určuje pohybové možnosti robota a jeho schopnosť vykonávať zložitejšie manipulačné úlohy.

1.2 Teoretické aspekty pohybu robotov

Aby sme pochopili, ako fungujú rôzne robotické systémy, je dôležité poznať ich kinematický a dynamický opis. Kinematika je oblasť mechaniky, ktorá opisuje pohyb telies v priestore bez uvažovania síl, zatiaľ čo dynamika tieto sily a momenty zohľadňuje [1, 3].

1.2.1 Kinematika robota

Kinematika opisuje geometrické vzťahy pohybu robota, teda najmä polohu, rýchlosť a zrýchlenie jednotlivých častí sústavy bez explicitného uvažovania síl. V robotike existujú dva prístupy – priama a spätná kinematika. Pre ich pochopenie je dôležité vedieť, že polohu možno opísať jednoduchým vektorom:

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (1)$$

Potom úloha priamej kinematiky určuje polohu koncového bodu, napríklad koncového efektora manipulátora, v zvolenom súradnicovom priestore. Výsledná poloha sa získa postupným násobením homogénnych transformačných matic:

$$\mathbf{T} = \mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \dots \cdot \mathbf{T}_n \quad (2)$$

kde každá matica $\mathbf{T}_i$ predstavuje transformáciu, teda posun (transláciu) alebo otočenie (rotáciu) medzi jednotlivými článkami robota [3]. Maticu translácie možno zapísať v tvare:

$$\mathbf{T}_{\text{trans}} = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

A matice rotácie nasledovne:

$$\mathbf{T}_{\text{rot}_x} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

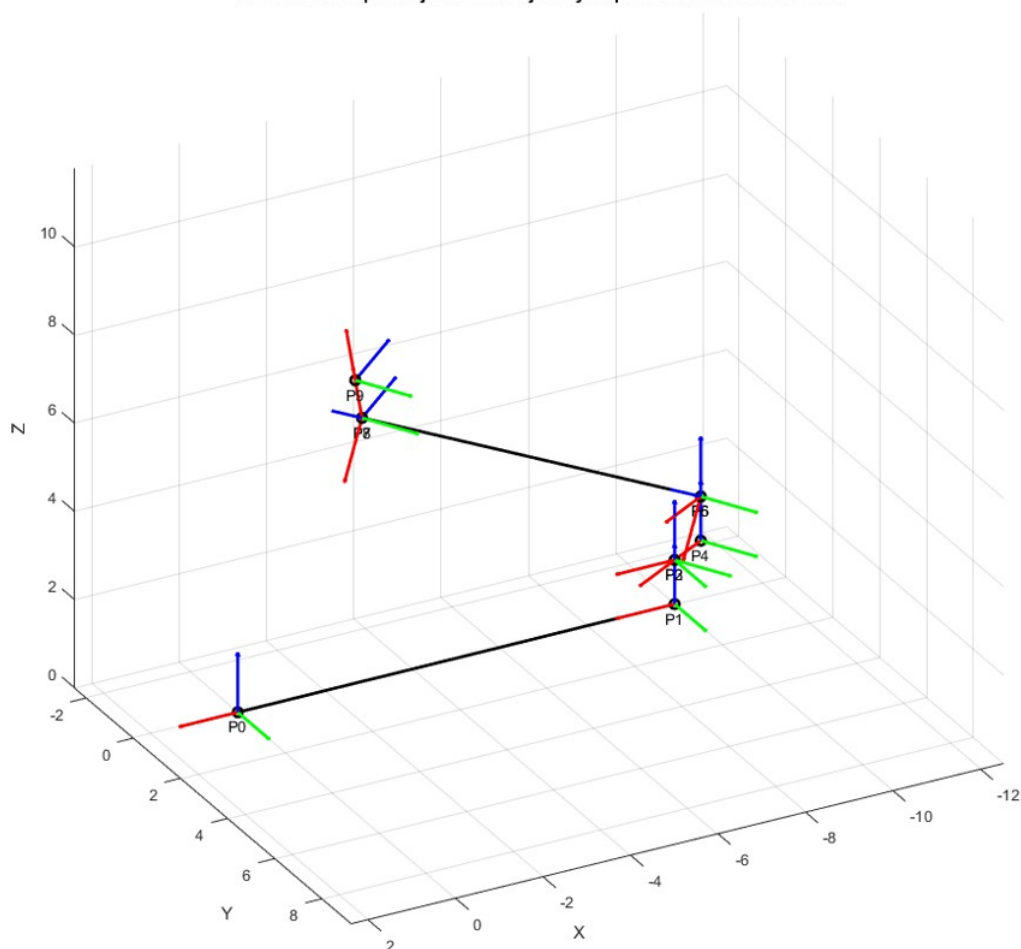
$$\mathbf{T}_{\text{rot}_y} = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

$$\mathbf{T}_{\text{rot}_z} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Ak poznáme dĺžku ramena, predlaktia a zápästia a príslušné kĺbové uhly, môžeme určiť polohu koncového bodu. Odpoveďou bude vektor, pretože všetky vynásobené matice ešte vynásobíme vektorom:

$$\mathbf{p}_{\text{konc}} = \mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \mathbf{T}_3 \cdot \mathbf{p}_{\text{start}} \quad (7)$$

Na obrázku nižšie je uvedený ilustračný príklad priamej kinematiky na modeli hasičského vozidla s výsuvným rebríkom. Bod P_0 predstavuje počiatočnú polohu, zatiaľ čo bod P_9 je konečná poloha koncového efektora, v tomto prípade koša na vrchu rebríka. Pomocou transformačných matic, ktoré zohľadňujú dĺžky jednotlivých segmentov a uhly ich natočenia, možno vypočítať presnú polohu koša v priestore.



Obr. 1: Príklad priamej kinematickej úlohy

Spätná kinematika rieši opačný problém – na základe požadovanej polohy koncového efektora určí, aké uhly by mali byť nastavené na jednotlivých kĺboch robota. Jednoducho povedané, chceme zdvihnúť pohár zo stola rukou a poznáme polohu pohára, v takom prípade sa rozhodujeme, pod akými uhlami a pohybmi môžeme dosiahnuť cieľ[1].

1.2.2 Dynamika robota

Dynamika robota sa zaoberá štúdiom síl a momentov, ktoré ovplyvňujú pohyb robotických systémov. Na rozdiel od kinematiky, ktorá sa zameriava na opis pohybu bez ohľadu na príčiny, dynamika skúma, ako tieto pohyby vznikajú v dôsledku pôsobenia vonkajších a vnútorných síl. V 18. storočí Leonard Euler a Joseph-Louis Lagrange formalizovali Eulerovu-Lagrangeovu rovnicu, ktorá sa dodnes používa pri modelovaní dynamických systémov:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = Q_i \quad (8)$$

kde L je Lagrangián systému, definovaný ako rozdiel medzi kinetickou energiou E_k a potenciálnou energiou E_p :

$$L = E_k - E_p \quad (9)$$

Q_i predstavuje všeobecnú silu pôsobiacu na súradnicu q_i . Táto rovnica umožňuje odvodiť pohybové rovnice robotických systémov na základe ich energetických vlastností[3].

1.3 Výzvy a trendy vývoja

Robotika sa v súčasnosti rozvíja na rozhraní mechaniky, riadenia, informatiky a umelej inteligencie [1]. Dynamický rozvoj tejto oblasti umožňuje nasadzovanie robotických systémov v priemysle, logistike, zdravotníctve, autonómnej doprave či službách pre domácnosti. Napriek výraznému technologickému pokroku však robotické systémy stále čelia viacerým technickým a praktickým obmedzeniam, ktoré komplikujú ich širšie nasadenie v reálnych podmienkach.

K hlavným obmedzeniam patrí energetická náročnosť robotických platforiem, najmä pri mobilných robotoch a autonómnych systémoch závislých od batériového napájania. Obmedzená kapacita zdrojov energie výrazne vplýva na dobu prevádzky, mobilitu aj výpočtové možnosti systému. Ďalším problémom je rastúca zložitosť riadenia pri vyššom počte stupňov voľnosti, kde sa zvyšujú nároky na presnosť modelovania, stabilitu riadenia a výpočtový výkon. Pri komplexných robotických manipulátoroch alebo humanoidných robotoch je navyše potrebné riešiť koordináciu viacerých pohybových článkov v reálnom čase.

Na druhej strane robotika ponúka významné perspektívy ďalšieho rozvoja. Významný pokrok prináša vývoj nových materiálov, ktoré umožňujú konštrukciu ľahších, odolnejších a energeticky efektívnejších robotických systémov. Rozvoj senzoriky zvyšuje presnosť percepcie prostredia a umožňuje detailnejšie spracovanie informácií v reálnom čase. Veľký význam má aj integrácia metód strojového učenia a umelej inteligencie, ktoré umožňujú robotom adaptívne správanie, učenie zo skúseností a efektívnejšie rozhodovanie v dynamických situáciách.

2 Plánovanie pohybu robotov

Plánovanie pohybu robotov predstavuje fundamentálnu úlohu robotiky, ktorá rieši spôsob, akým sa robot dostane do požadovaného bodu alebo konfigurácie. Táto časť sa zaoberá základnými poznatkami v tejto oblasti, vybranými algoritmami a kritériami ich hodnotenia, ktoré budú dôležité pre ďalšiu analýzu [1, 4].

2.1 Hlavné úlohy plánovania pohybu

Hlavné úlohy plánovania pohybu možno rozdeliť do viacerých kategórií podľa toho, na aký aspekt sa problém zameriava. V podstate ide o vytvorenie trajektórie alebo sekvencie príkazov, ktoré prevezmú robota z počiatočného stavu do cieľového stavu, pričom sa vyhýbajú prekážkam a často optimalizujú nejaké kritérium.

2.1.1 Plánovanie trasy a plánovanie trajektórie

Ako bolo spomenuté vyššie, dynamika zohľadňuje sily a energiu pôsobiace na robota. Samotné plánovanie trasy (z angl. *path planning*) sa tiež líši od plánovania trajektórie (z angl. *trajectory planning*) [1]. Plánovanie cesty možno chápať ako určenie sekvencie bodov vedúcich od počiatočného k cieľovému stavu. Na druhej strane plánovanie trajektórie už zohľadňuje aj dynamické aspekty pohybu, ako sú rýchlosť, zrýchlenie a časovanie jednotlivých úsekov [1].

2.1.2 Konfiguračný priestor

Konfiguračný priestor (z angl. *C-space*, *Configuration Space*) predstavuje množinu všetkých možných konfigurácií robota. Konfigurácia q je vektor, ktorý opisuje polohu a orientáciu každého DOF robota, teda $q = (q_1, q_2, \dots, q_n)$. Ich počet n určuje dimenziu priestoru C . Napríklad diferenciálny robot má 3 DOF ($x, y, \theta \in SE(2)$), kvadrokoptéra 6 DOF ($p, R \in SE(3)$) a manipulačné rameno so siedmimi kĺbmi má 7 DOF ($\theta_1, \dots, \theta_7 \in [-\pi, \pi)^7$). Každý bod v priestore C teda reprezentuje určitú polohu robota v pracovnom prostredí.

Hlavným prínosom konfiguračného priestoru je, že umožňuje transformovať problém plánovania pohybu robota z reálneho pracovného priestoru do abstraktného matematického priestoru, v ktorom sa pohyb robota modeluje ako pohyb bodu. Tým sa zložité problémy s kolíziami a geometriou robota redukujú na problém hľadania trajektórie bodu v priestore s prekážkami [5].

Každá konfigurácia môže byť buď voľná, alebo kolízna. Ak sa robot v konfigurácii q nedotýka žiadnej prekážky, hovoríme, že táto konfigurácia je voľná. Množina všetkých

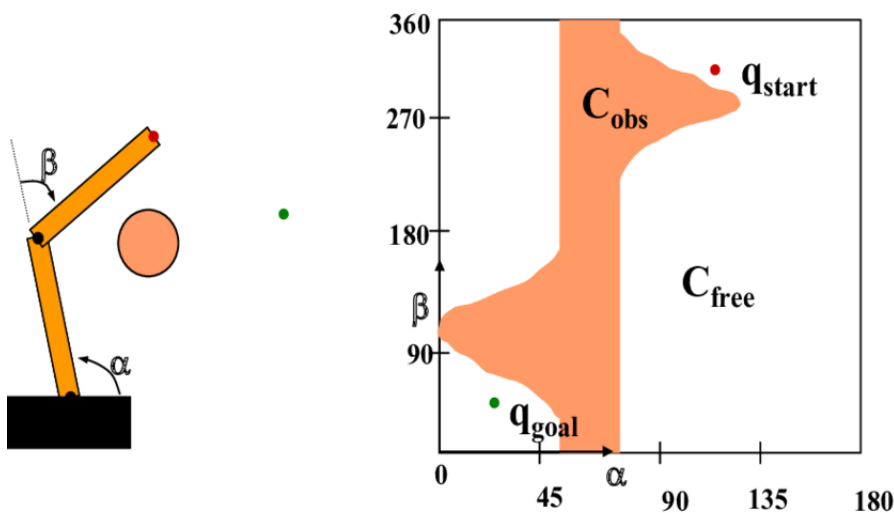
takýchto konfigurácií tvorí *voľný priestor*:

$$C_{free} = \{q \in C \mid \text{robot v } q \text{ neinteraguje s prekážkami}\}. \quad (10)$$

Naopak, množina konfigurácií, v ktorých robot koliduje s prekážkami alebo so sebou samým, sa nazýva *obsadený priestor*:

$$C_{obs} = \{q \in C \mid \text{robot v } q \text{ je v kolízii}\}. \quad (11)$$

Na obrázku nižšie je uvedený príklad C-priestoru pre robota s dvoma kĺbmi.



Obr. 2: C_{space} , C_{free} a C_{obs} pre artikulovaného robota s dvoma kĺbmi [6].

V praxi sa plánovanie pohybu v konfiguračnom priestore často rieši tak, že sa konštruuje graf, ktorého uzly predstavujú konfigurácie v C_{free} a hrany predstavujú možné prechody medzi nimi. Takýto graf možno zostaviť rôznymi spôsobmi: pomocou pravidelnej mriežky, adaptívnej dekompozície, alebo pomocou metód ako sú grafy viditeľnosti, Voronoi diagramy, či pravdepodobnostné cestovné mapy. Po vytvorení grafu sa úloha plánovania pohybu redukuje na úlohu hľadania najkratšej cesty medzi uzlami q_I a q_G v grafe.

Konfiguračný priestor teda poskytuje všeobecný a unifikovaný rámec pre formuláciu problému plánovania pohybu. Jeho výhodou je zjednodušenie analýzy na úroveň bodového robota, zatiaľ čo nevýhodou je vysoká výpočtová náročnosť, najmä pre roboty s veľkým počtom stupňov voľnosti, kde dimenzia priestoru rýchlo rastie [4, 7].

2.1.3 Hlavné ciele štúdia

Cieľom plánovania pohybu je nájsť trajektóriu, ktorá dovedie robota do cieľovej konfigurácie bez kolízie a zároveň spĺňa zvolené optimalizačné kritériá [8]. Medzi najdôležitejšie

patrí bezpečnosť pohybu v priestore C_{free} , čas vykonania, energetická náročnosť a plynulosť trajektórie [1, 9]. Tieto požiadavky sú často vo vzájomnom konflikte, preto sa v praxi hľadá kompromis medzi bezpečnosťou, efektivitou a kvalitou pohybu.

2.2 Klasické metódy plánovania pohybu

Klasické metódy plánovania pohybu, niekedy označované aj ako metódy založené na modeli, predstavujú fundamentálny pilier robotiky. Na rozdiel od moderných prístupov sa spoliehajú na explicitný geometrický a/alebo dynamický model prostredia a robota. Ich hlavným cieľom je nájsť trajektóriu z počiatočnej konfigurácie do cieľovej konfigurácie, ktorá je realizovateľná, teda vyhýba sa prekážkam a rešpektuje obmedzenia systému, a zároveň optimalizuje zvolené kritérium, napríklad dĺžku dráhy, čas pohybu alebo spotrebu energie [1, 8]. Klasické metódy možno rozdeliť na grafové metódy a metódy priamej optimalizácie v spojitom priestore trajektórií.

2.2.1 Vyhľadávanie v grafoch

Tieto algoritmy operujú na diskretnej reprezentácii priestoru, najčastejšie v podobe mriežky alebo grafu. Hlavnou myšlienkou je prehľadávať množinu možných konfigurácií (uzlov grafu) s cieľom nájsť cestu s minimálnymi nákladmi.

- **Dijkstrov algoritmus** je základný grafový algoritmus, ktorý nájde najkratšiu cestu z jedného počiatočného uzla do všetkých ostatných uzlov v grafe s nezápornými hranovými váhami. Funguje na princípe postupného rozširovania uzlov podľa ich aktuálnej nákladovej funkcie $g(n)$. Hoci garantuje optimálne riešenie, môže byť výpočtovo neefektívny, pretože prehľadáva veľkú časť stavového priestoru bez explicitného smerovania k cieľu [10].
- **A*** je vylepšením Dijkstrovho algoritmu, ktoré zavádza heuristickú funkciu $h(n)$ pre odhad nákladov z uzla n do cieľa. Algoritmus expanduje uzly podľa hodnoty funkcie $f(n) = g(n) + h(n)$, kde $g(n)$ sú skutočné náklady zo štartu do n a $h(n)$ je heuristický odhad. Ak je heuristika *prípustná* a *konzistentná*, A* garantuje nájdenie optimálnej cesty a zvyčajne prehľadáva menší počet stavov než Dijkstrov algoritmus [10].
- **Theta*** je rozšírením A* pre mriežkové prostredia, ktoré odstraňuje typickú zalomenosť trajektórie vznikajúcu pri pohybe viazanom na hrany mriežky. Ak existuje priama línia bez prekážok medzi predchodcom aktuálneho uzla a jeho susedom, Theta* vytvorí hranu medzi týmito uzlami, čím výslednú dráhu vyhladí a často aj skráti [11].

- **D* Lite** patrí medzi inkrementálne replánovacie algoritmy. Je vhodný pre situácie, v ktorých sa mapa prostredia mení alebo sa postupne spresňuje. Algoritmus efektívne upravuje už vypočítaný plán po objavení nových prekážok, pričom využíva výsledky predchádzajúceho vyhľadávania [12].

2.2.2 Optimalizačné metódy

Tieto metódy považujú plánovanie trajektórie za problém spojitej optimalizácie. Namiesto vyhľadávania v diskretnom grafe priamo optimalizujú parametrizovanú trajektóriu ξ s cieľom minimalizovať nákladovú funkciu, ktorá zohľadňuje plynulosť, bezpečnosť a dynamické obmedzenia [13, 14].

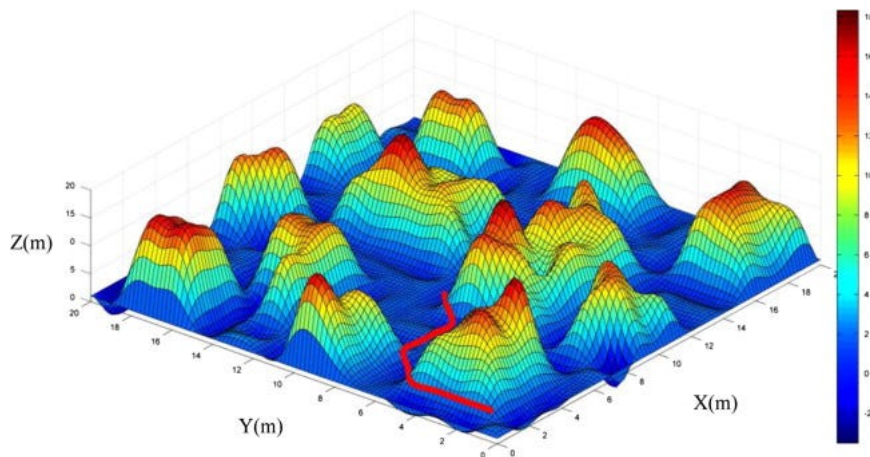
- **CHOMP** (z angl. *Covariant Hamiltonian Optimization for Motion Planning*) formuluje problém plánovania ako optimalizáciu v priestore trajektórií. Metóda používa funkcionálny gradient a metriku odvodenú z dynamiky systému, čím podporuje rýchlu konvergenciu k hladkej a bezpečnej trajektórii [13].
- **STOMP** (z angl. *Stochastic Trajectory Optimization for Motion Planning*) na rozdiel od CHOMP nevyžaduje analytický výpočet gradientu nákladovej funkcie. Namiesto toho generuje množstvo náhodných variácií okolo aktuálnej trajektórie a novú trajektóriu určuje ako váženú kombináciu skúmaných vzoriek [14].
- **TrajOpt** (*Trajectory Optimization*) formuluje problém optimalizácie trajektórie ako sekvenciu nelineárnych optimalizačných problémov s obmedzeniami. Kolízie modeluje explicitne ako nerovnostné obmedzenia a na riešenie využíva metódu sekvenčného kvadratického programovania [15].

2.2.3 Metódy potenciálneho poľa

Metódy potenciálneho poľa predstavujú intuitívny a výpočtovo efektívny prístup k plánovaniu dráhy v reálnom čase, ktorý je obzvlášť vhodný pre reakčné vyhýbanie sa prekážkam. Základná idea spočíva v konštrukcii virtuálneho poľa síl v konfiguračnom priestore robota [16].

Na obrázku vyššie cieľ je reprezentovaný modrým bodom, prekážky červenými bodmi a silové vektory ukazujú smer pohybu robota. Cieľovej polohe je priradený *atraktívny potenciál* $U_{\text{att}}(\mathbf{q})$, ktorý robota priťahuje, zatiaľ čo prekážkam je priradený *repulzívny potenciál* $U_{\text{rep}}(\mathbf{q})$, ktorý robota odpudzuje. Výsledné pole je definované ako súčet týchto príspevkov [16]:

$$U_{\text{total}}(\mathbf{q}) = U_{\text{att}}(\mathbf{q}) + U_{\text{rep}}(\mathbf{q}) \quad (12)$$



Obr. 3: Ilustrácia potenciálneho poľa pre plánovanie pohybu [16]

Robot sa potom pohybuje v smere záporného gradientu tohto celkového potenciálu, čím vlastne "klesá" po energetickom svahu smerom k cieľu. Výsledná sila pôsobiaca na robota je daná ako:

$$\mathbf{F}_{\text{total}}(\mathbf{q}) = -\nabla U_{\text{total}}(\mathbf{q}) = \mathbf{F}_{\text{att}}(\mathbf{q}) + \mathbf{F}_{\text{rep}}(\mathbf{q}) \quad (13)$$

Kde $\mathbf{F}_{\text{att}}$ je atraktívna sila smerujúca k cieľu a $\mathbf{F}_{\text{rep}}$ je repulzívna sila odpudzujúca od prekážok.

Hlavnou výhodou tohto prístupu je jednoduchosť, nízka výpočtová náročnosť a schopnosť generovať trajektóriu priamo v spojitom priestore. Vďaka tomu je metóda vhodná najmä pre lokálne plánovanie a rýchlu reakciu na prekážky.

Metóda umelých potenciálnych polí má aj viacero známych obmedzení. V komplexných priestoroch môže výsledné potenciálne pole obsahovať lokálne minimá, do ktorých robot môže „spadnúť“ a zostať uväznený bez možnosti dosiahnuť globálne minimum reprezentujúce cieľ. Ďalším problémom sú oscilácie v úzkych priechodoch medzi prekážkami, kde dochádza k rýchlemu striedaniu atraktívnych a repulzívnych síl, čo môže viesť k nestabilnému alebo neefektívnemu pohybu robota. Nevýhodou je aj skutočnosť, že výsledná trajektória nie je z globálneho hľadiska nutne optimálna, keďže pohyb robota je založený iba na lokálnom gradientovom zostupe potenciálneho poľa.

Napriek obmedzeniam sa metódy potenciálneho poľa často používajú ako lokálny plánovač v hybridných architektúrach alebo pre jednoduché úlohy vyhýbania sa prekážkam.

2.2.4 Metódy vzorkovania

Metódy založené na vzorkovaní stavového priestoru sú navrhnuté tak, aby efektívne riešili problém plánovania pohybu pre roboty s veľkým počtom stupňov voľnosti. Na-

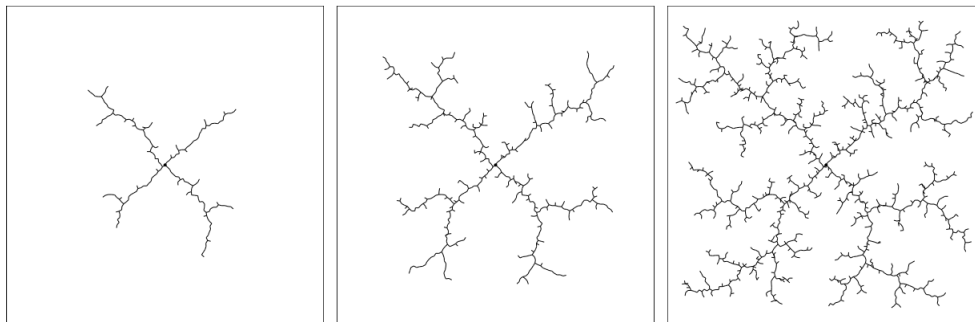
miesto explicitnej diskretizácie celého priestoru náhodne generujú vzorky konfigurácií a pokúšajú sa z nich vybudovať grafovú štruktúru reprezentujúcu realizovateľný priestor [17, 18].

- **PRM** (z angl. *Probabilistic Roadmap*) je metóda vhodná pre plánovanie viacerých dotazov v statickom prostredí. Algoritmus pozostáva z dvoch fáz [17]:

1. **Fáza konštrukcie roadmapu:** V konfiguračnom priestore sa náhodne generujú vzorky (konfigurácie). Pre každú realizovateľnú vzorku, teda takú, ktorá nie je v kolízii, sa prehľadávaním v okolí (napr. k-najbližších susedov) nájdu susedné realizovateľné konfigurácie a spoja sa s nimi hranou, pokiaľ je medzi nimi priama kolízne voľná dráha. Tým sa vytvorí graf (roadmap) pretínajúci realizovateľný priestor.
2. **Fáza dotazu:** Pre daný počiatočný a cieľový bod sa tieto body pridajú do roadmapu a prepoja s existujúcimi uzlami. Následne sa v grafe vyhľadá dráha pomocou algoritmov ako A* alebo Dijkstra.

PRM je veľmi efektívny pre roboty s vysokou dimenziou, pretože nevyžaduje vyplnenie celého priestoru, ale vytvára jeho účinnú aproximáciu.

- **RRT** (z angl. *Rapidly-exploring Random Tree*) je metóda navrhnutá pre plánovanie jedného dotazu, teda pre nájdenie jednej cesty medzi konkrétnym počiatočným a cieľovým bodom. Algoritmus iteratívne rozširuje strom, ktorý je zakorenený v počiatočnej konfigurácii [19].



Obr. 4: Vzrastajúci v čase RRT [19].

1. Náhodne sa vygeneruje vzorka $\mathbf{q}_{\text{rand}}$ v konfiguračnom priestore.
2. Nájde sa uzol $\mathbf{q}_{\text{near}}$ v strome, ktorý je najbližšie k $\mathbf{q}_{\text{rand}}$.
3. $\mathbf{q}_{\text{near}}$ sa pokúsi vygenerovať nový uzol $\mathbf{q}_{\text{new}}$ posunutím o malý krok (s fixnou dĺžkou) v smere k $\mathbf{q}_{\text{rand}}$.

4. Ak je hrana medzi $\mathbf{q}_{\text{near}}$ a $\mathbf{q}_{\text{new}}$ realizovateľná, pridá sa $\mathbf{q}_{\text{new}}$ do stromu.

Tento proces sa opakuje, kým nie je $\mathbf{q}_{\text{new}}$ dostatočne blízko cieľu alebo nie je prekročený maximálny počet iterácií.

Hlavnou prednosťou RRT je jeho schopnosť rýchlo prehľadávať rozsiahle priestory a dobre škálovať s dimenziou. Štandardný RRT však negarantuje optimalitu dráhy. Toto obmedzenie riešia jeho pokročilejšie varianty:

- **RRT***: Prináša garantovanú asymptotickú optimalitu. Po pridaní nového uzla $\mathbf{q}_{\text{new}}$ sa pre jeho okolie hľadá, či by neexistovalo "lepšie" rodičovské spojenie, ktoré by poskytlo nižšie celkové náklady cesty zo štartu. Zároveň sa pre existujúce uzly v okolí vykoná "rewiring", čím sa strom neustále zlepšuje.
- **RRT-Connect**: Zvyšuje rýchlosť konvergenzie tým, že súčasne rastie jeden strom od štartu a druhý od cieľa a snaží sa ich spojiť [18].

Metódy vzorkovania, najmä RRT a jeho varianty, patria medzi často používané prístupy pri plánovaní pohybu robotických manipulátorov a humanoidných robotov, najmä vďaka svojej efektívnosti a schopnosti pracovať aj vo vysokodimenzionálnych priestoroch [7, 18].

3 Neurónové siete a ich využitie v plánovaní pohybu

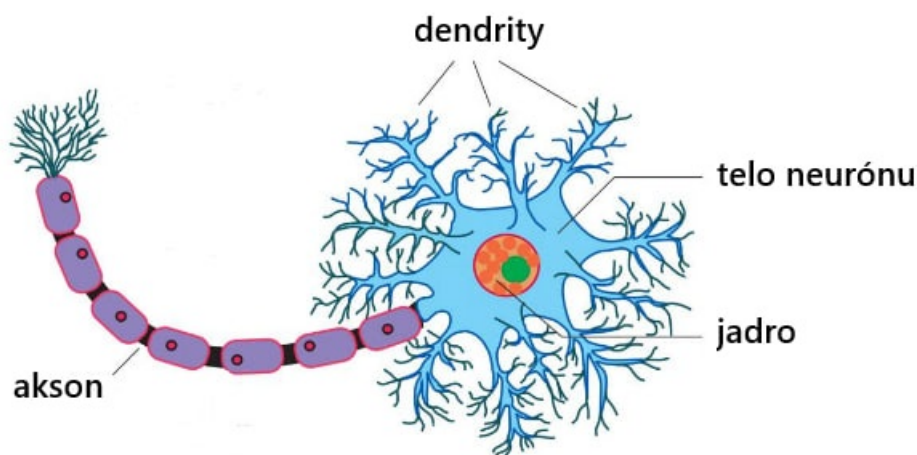
V tejto kapitole sa zaoberáme základmi neurónových sietí, od porovnania umelého neurónu s biologickým až po základné architektúry sietí používané pri riadení a učení robotických systémov. Osobitná pozornosť sa venuje učeniu s posilňovaním, ktoré sa používa na riadenie 7-osového robotického manipulátora.

3.1 Základy neurónových sietí

Neurónová sieť je výpočtový systém pozostávajúci z jednoduchých prvkov – neurónov. Inšpirované biologickým mozgom, umelé neurónové siete predstavujú určitý matematický model toho, ako neuróny v mozgu spracúvajú informácie. Jednoducho povedané, neurónová sieť je súbor prepojených výpočtových jednotiek (teda umelých neurónov), ktoré spolupracujú na riešení zložitých úloh, ako je rozpoznávanie obrazov, klasifikácia rôznych údajov, prognózovanie a plánovanie pohybu [20].

3.1.1 Biologická analógia a matematický model neurónu

Na obrázku nižšie je znázornená štruktúra biologického neurónu, ktorý je základnou stavebnou jednotkou nervového systému. Podľa článku *‘The Neuron’* od Kandela, Schwartz a Jessella [21], ktorý poskytuje komplexný prehľad neurobiológie, sú kľúčové časti neurónu a ich funkcie nasledovné:

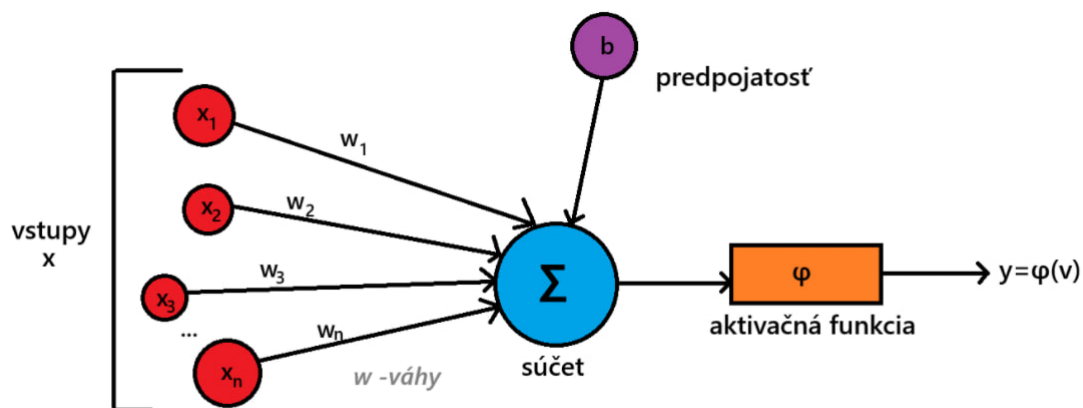


Obr. 5: Štruktúra biologického neurónu [22]

- Dendrity: Slúžia na príjem elektrochemických signálov od iných neurónov cez špeciálne spojenia nazývané **synapsy**.
- Telo neurónu: Integruje prichádzajúce signály. Ak je celková úroveň excitácie dostatočne vysoká, neurón generuje výstupný signál.
- Axon: Predĺženie neurónu, ktoré vedie vzniknutý elektrický signál (akčný potenciál) smerom k ďalším neurónom.
- Synapse: Spojenie medzi axonom jedného neurónu a dendritom druhého, kde chemický neurotransmitter prenáša signál.

Princíp činnosti spočíva v tom, že vstupy prichádzajúce do dendritov sú váhované (synaptická sila) a integrované v tele neurónu. Ak integrovaný signál prekročí určitú prahovú hodnotu, neurón sa *‘aktivuje’* a vyšle signál axonom ďalej do siete.

Tento biologický princíp inšpiroval vytvorenie matematického modelu neurónu pre umelé neurónové siete. Podľa Gurneyho [20] je najjednoduchší model formálny neurón, ktorý:



Obr. 6: Matematický model neurónu

1. Prijme vstupný vektor $\mathbf{x} = (x_1, x_2, \dots, x_n)$, kde každá zložka reprezentuje signál z iného neurónu.
2. Každému vstupu x_i priradí synaptickú váhu w_i (z angl. *weight*), ktorá reprezentuje silu spojenia.
3. Vypočíta vážený súčet všetkých vstupov: $u = \sum_{i=1}^n w_i x_i$.

4. K tomuto súčtu sa často pripočíta ešte biasový člen b , ktorý posúva aktivačnú funkciu: $v = u + b$.
5. Výsledná hodnota v sa potom aplikuje na aktivačnú funkciu $\varphi(\cdot)$, čím sa získa výstup neurónu: $y = \varphi(v)$.

Najzákladnejšou formou aktivačnej funkcie je *prahová funkcia* (z angl. *Heaviside step function*), kde neurón generuje výstup 1, ak $v \geq 0$, inak 0. Toto jednoduché porovnanie priamo zodpovedá biologickému konceptu prahovej hodnoty pre aktiváciu neurónu. Takýto model, hoci veľmi zjednodušený, tvorí základ pre perceptrón a komplexnejšie architektúry neurónových sietí.

3.1.2 Aktivačné funkcie v neurónových sieťach

Aktivačné funkcie predstavujú nelineárne mapovania, ktoré prevádzajú vstupné hodnoty neurónov na výstupné signály. Bez ich použitia by viacvrstvová neurónová sieť bola len lineárnou kombináciou váh, a teda by nebola schopná modelovať zložité nelineárne vzťahy. Vďaka aktivačným funkciám môžu siete učiť nelineárne hranice a aproximovať zložitejšie funkcie [20, 23].

Medzi najpoužívanéjšie patrí **sigmoidálna** funkcia,

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (14)$$

ktorá transformuje reálne hodnoty do intervalu $(0, 1)$. Je vhodná pre modely, kde sa výstupy interpretujú ako pravdepodobnosti. Nevýhodou sigmoidálnej funkcie je saturácia (výstup sa blíži k 0 alebo 1), čo spôsobuje miznúci gradient a spomaľuje učenie hlbokých sietí.

Ďalšou často používanou funkciou je **hyperbolický tangens**,

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}, \quad (15)$$

ktorý má výstup v rozsahu $(-1, 1)$ a poskytuje nulové stredné hodnoty výstupov, čo zlepšuje konvergenciu učenia oproti sigmoidálnej funkcii. Napriek tomu sa aj $\tanh$ môže nasýtiť a trpieť stratou gradientu pri veľkých hodnotách vstupu.

Najvýznamnejším pokrokom v oblasti aktivačných funkcií bolo zavedenie ReLU (z angl. *Rectified Linear Unit*),

$$f(x) = \max(0, x), \quad (16)$$

ktorá zachováva lineárnu odozvu pre kladné hodnoty a eliminuje saturáciu pre záporné vstupy. ReLU je výpočtovo efektívna, podporuje riedke aktivácie (mnohé neuróny majú výstup 0) a urýchľuje konvergenciu stochastického gradientného zostupu. Nevýhodou

je riziko tzv. mŕtvych neurónov, ktoré prestanú reagovať, ak sa ich váhy dostanú do oblasti so zápornými vstupmi.

Okrem klasických funkcií sa v moderných modeloch používajú aj modifikácie ako **Leaky ReLU** a **ELU** (Exponential Linear Unit), ktoré sa snažia zlepšiť tok gradientu a stabilitu učenia.

Výber aktivačnej funkcie závisí od typu úlohy, architektúry siete a rozsahu výstupov. Napríklad pri klasifikácii s viacnásobnými triedami sa často používa **softmax** funkcia v poslednej vrstve:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}, \quad (17)$$

ktorá prevádza výstupy neurónov na pravdepodobnostnú distribúciu.

3.1.3 Schopnosť generalizácie

Schopnosť generalizácie (z angl. *generalization ability*) označuje, do akej miery je model schopný správne reagovať na dáta, ktoré nevidel počas tréningu. Neurónová sieť s dobrými generalizačnými vlastnosťami sa naučí podstatu vzoru v dátach, nie len zapamätanie tréningových príkladov.

Generalizácia úzko súvisí s komplexnosťou siete, množstvom tréningových dát a použitými regularizačnými technikami. Príliš jednoduchý model má tendenciu k **podtrenovaniu** (z angl. *underfitting*), zatiaľ čo príliš komplexný model môže trpieť **pretrenovaním** (z angl. *overfitting*), teda stratou schopnosti reagovať na nové dáta.

Medzi metódy, ktoré zlepšujú generalizáciu, patria:

- **Dropout** — náhodné vypínanie neurónov počas tréningu, ktoré bráni závislosti medzi jednotkami.
- **Regularizácia L_1 a L_2** — penalizácia veľkých hodnôt váh, ktorá podporuje jednoduchosť modelu.
- **Batchová normalizácia** (z angl. *Batch Normalization*) — stabilizuje distribúciu aktivácií v sieti.
- **Augmentácia dát** (z angl. *data augmentation*) — rozširovanie tréningovej množiny o mierne pozmenené verzie dát.

Aktivačné funkcie tiež ovplyvňujú schopnosť generalizácie, pretože regulujú, aký typ nelinearity sa sieť učí. Napríklad ReLU podporuje riedku aktiváciu, čo často vedie k lepšiemu zovšeobecneniu. Naopak, saturujúce funkcie ako sigmoid môžu viesť k slabšiemu prenosu gradientu a horšiemu prispôsobeniu.

Schopnosť generalizácie teda závisí od rovnováhy medzi kapacitou modelu, regularizáciou a stabilitou procesu učenia [20].

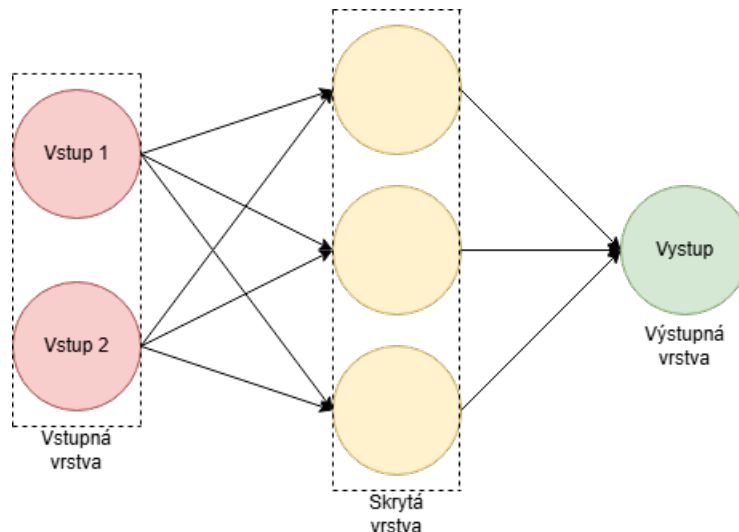
3.2 Architektúra neurónových sietí

Podľa typu architektúry môžeme NN rozdeliť na nasledujúce typy: jednovrstvové, viacvrstvové, konvolučné a rekurentné.

Ako bolo uvedené v predchádzajúcej podkapitole, perceptróny predstavujú základný model umelej neurónovej architektúry. Tento typ siete využíva prahovú (binárnu) aktivačnú funkciu a nachádza uplatnenie najmä v úlohách, ktoré sú *lineárne separovateľné*, ako napríklad jednoduché logické operácie (AND, OR) alebo základná binárna klasifikácia signálov. Ich hlavnou výhodou je výpočtová jednoduchosť, no zároveň sú obmedzené tým, že nedokážu riešiť nelineárne problémy (napr. XOR).

3.2.1 Viacvrstvové neurónové siete

Podľa Gurneyho [20] vznikli viacvrstvové NN (MLP – Multilayer Perceptron) práve ako reakcia na tieto obmedzenia. Viacvrstvová architektúra pridáva medzi vstupnú a výstupnú vrstvu jednu alebo viac *skrytých vrstiev*, ktoré umožňujú modelovať nelineárne vzťahy medzi vstupmi a výstupmi. Každý neurón v týchto vrstvách používa nelineárnu aktivačnú funkciu, ako napríklad sigmoid, tangens hyperbolický alebo ReLU, čím sa zvyšuje expresívna schopnosť siete.



Obr. 7: Viacvrstvová neurónová sieť (MLP)

Tréning viacvrstvových sietí prebieha pomocou algoritmu **spätnej propagácie chyby** (z angl. *backpropagation*), ktorý upravuje váhy všetkých vrstiev s cieľom mi-

nimalizovať rozdiel medzi požadovaným a skutočným výstupom. Tento mechanizmus umožňuje sieti postupne „učiť sa“ z množiny tréningových dát.

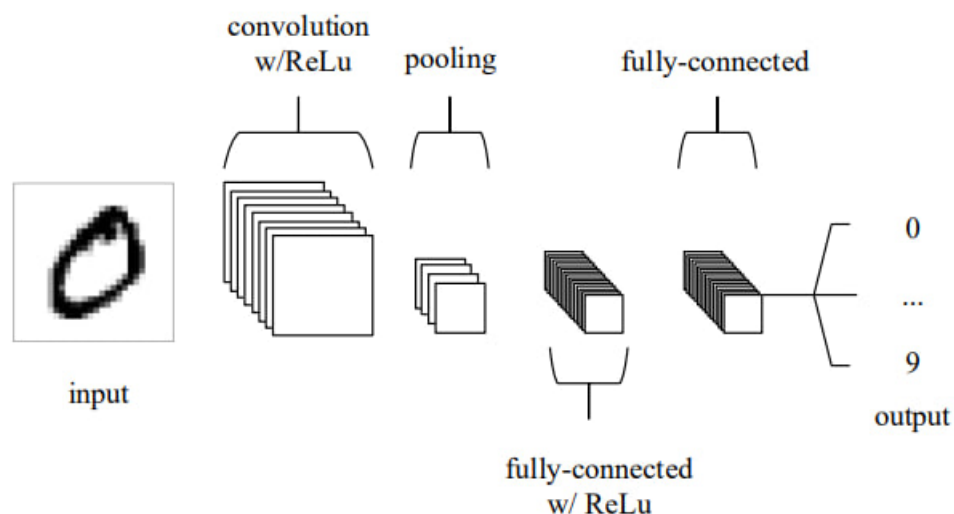
Z architektonického hľadiska teda jednovrstvové siete predstavujú jednoduchý, lineárny model vhodný pre základné úlohy rozpoznávania vzorov, zatiaľ čo viacvrstvové siete prinášajú nelineárne správanie a podstatne vyššiu expresívnu schopnosť. Ako uvádza Gurney, tieto dve architektúry tvoria základ dopredných (z angl. *feed-forward*) neurónových sietí, ktoré sú východiskom pre modernejšie modely hlbokého učenia [20].

Vo všetkých neurónových sieťach sa pri učení používajú stratové funkcie, pričom medzi najčastejšie používané patria [20]:

- Stredná kvadratická odchýlka alebo MSE (z angl. *Mean Squared Error*) - pri aproximácii, kde sa snažíme minimalizovať priemerný štvorcový rozdiel medzi predikovanými a skutočnými hodnotami.
- Binárna krížová entropia alebo BCE (z angl. *Binary Cross-Entropy*) - pri klasifikačných úlohách, kde sa snažíme minimalizovať rozdiel medzi pravdepodobnostnou distribúciou predpovede a skutočnou distribúciou tried.

3.2.2 Konvolučné neurónové siete

Konvolučné neurónové siete (ďalej len CNN, z angl. *Convolutional Neural Networks*) sú architektúry určené najmä na spracovanie obrazových dát [24]. Ako je znázornené na obrázku nižšie, CNN sa skladá z troch základných typov vrstiev, ak neberieme do úvahy vstup a výstup:



Obr. 8: CNN na rozpoznávanie napísaných čísel [24]

- Konvolučná vrstva (z angl. *convolution layer*), ktorá je prepojená s lokálnymi oblasťami vstupu prostredníctvom výpočtu skalárneho súčinu medzi ich váhami a oblasťou súvisiacou s objemom vstupu. ReLU sa v tejto vrstve často používa ako aktivačná funkcia aplikovaná na výstup predchádzajúcej vrstvy.
- Vrstva zlučovania (z angl. *pooling layer*) jednoducho vykoná zníženie diskretizácie podľa priestorovej dimenzie zadaného vstupu, čím sa ešte viac zníži počet parametrov v rámci tejto aktivácie.
- Plne prepojené vrstvy (z angl. *fully connected layers*) plnia rovnaké funkcie ako v štandardných NN a snažia sa generovať odhady tried na základe aktivácií, ktoré sa použijú na klasifikáciu. V praxi sa medzi vrstvami často používa aktivačná funkcia ReLU.

V kontexte tejto práce majú CNN predovšetkým doplňujúci význam. V praktickej časti sa priamo nevyužívajú, no uvádzajú sa ako súčasť širšieho prehľadu základných architektur neurónových sietí.

3.2.3 Rekurentné neurónové siete

Rekurentné neurónové siete (z angl. *Recurrent Neural Networks*, ďalej len RNN) predstavujú triedu neurónových sietí určených na spracovanie sekvenčných dát, teda údajov, kde má poradie prvkov zásadný význam. Na rozdiel od klasických dopredných sietí predpokladajúcich nezávislé vstupy obsahujú RNN spätné väzby, vďaka ktorým si môžu „pamätať“ predchádzajúce stavy. Tento mechanizmus im umožňuje modelovať časové závislosti a dynamické procesy [20].

Základná architektúra RNN pozostáva z troch vrstiev:

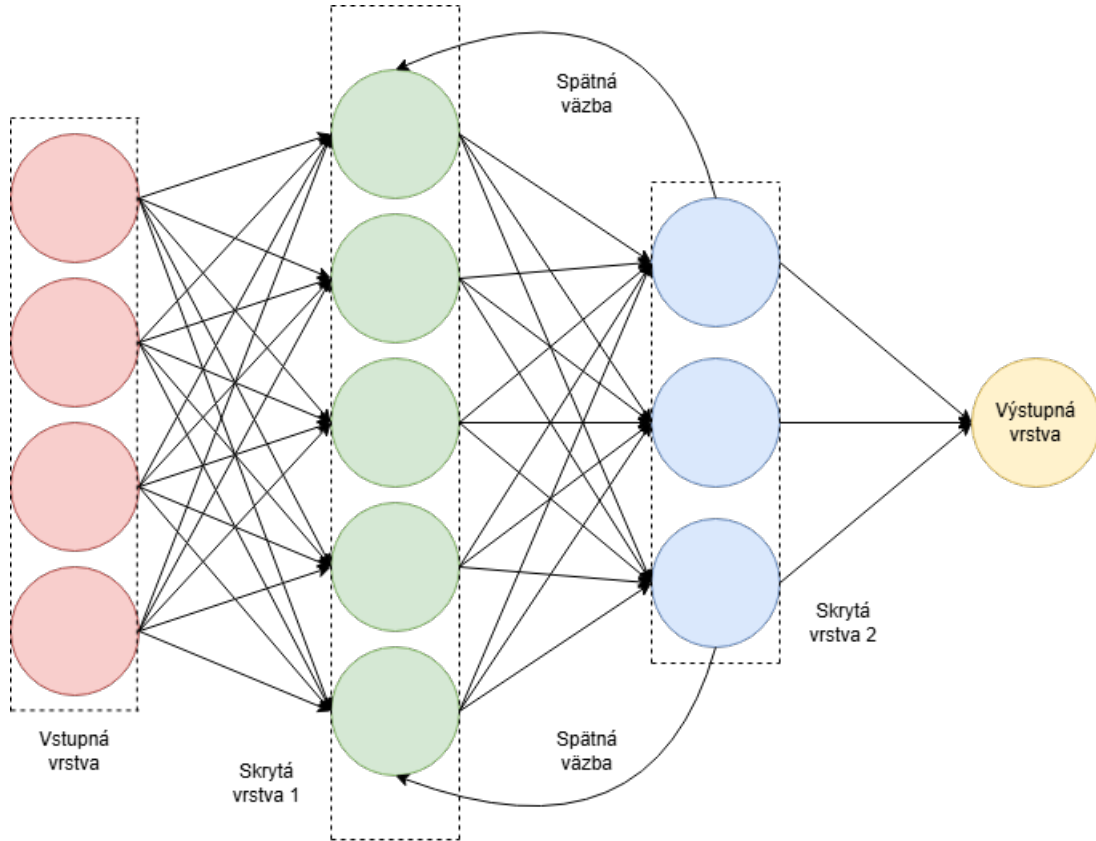
1. vstupná vrstva $\mathbf{x}_t$, ktorá prijíma dáta v časovom kroku t ,
2. skrytá vrstva $\mathbf{h}_t$, ktorá kombinuje aktuálny vstup a predchádzajúci stav,
3. výstupná vrstva $\mathbf{y}_t$, ktorá generuje odpoveď siete.

Aktualizácia skrytého stavu sa vykonáva podľa rovnice

$$\mathbf{h}_t = f_H(W_{IH}\mathbf{x}_t + W_{HH}\mathbf{h}_{t-1} + \mathbf{b}_h), \quad (18)$$

kde f_H je aktivačná funkcia (napr. tanh alebo ReLU), W_{IH} sú váhy medzi vstupnou a skrytou vrstvou a W_{HH} predstavuje rekurentné prepojenia medzi stavmi. Výstup siete je potom daný

$$\mathbf{y}_t = f_O(W_{HO}\mathbf{h}_t + \mathbf{b}_o). \quad (19)$$



Obr. 9: Schéma rekurentnej neurónovej siete

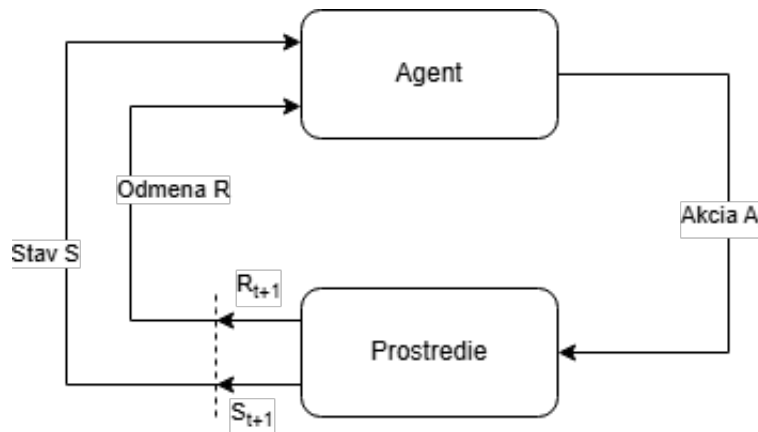
Tento model je možné rozvinúť v čase (z angl. *unroll*), čím sa rekurentná sieť transformuje na sekvenciu vrstiev so zdieľanými váhami. Tréning RNN sa vykonáva pomocou metódy spätnej propagácie v čase (z angl. *backpropagation through time*, BPTT), ktorá spätne šíri chybu cez časové kroky. Rovnako ako pri CNN, aj RNN sú v tejto práci uvedené najmä pre doplnenie širšieho kontextu neurónových architektúr. V experimentálnej časti sa priamo nehodnotia, keďže praktická implementácia je založená na algoritmoch RL využívajúcich iný typ politiky a hodnotových sietí.

3.3 Učenie s posilňovaním

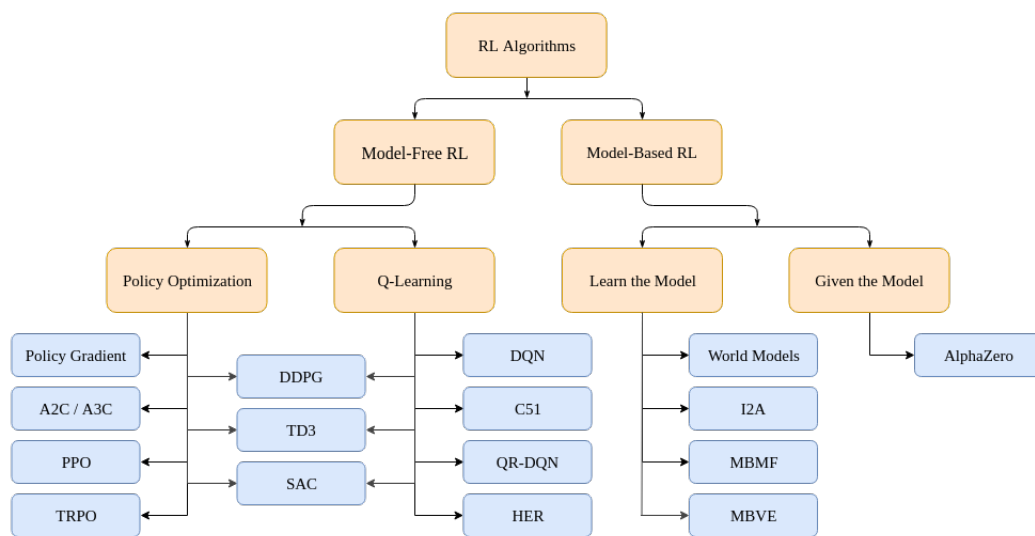
Ako paradigma strojového učenia, RL je rámec založený na skúmaní (z angl. *exploration*) a využívaní (z angl. *exploitation*), pričom cieľom je maximalizovať súčet budúcich diskontovaných odmien[25].

Na obrázku vyššie je znázornená všeobecná schéma učenia s posilňovaním. Formálne možno RL definovať ako optimalizačný problém, v ktorom agent hľadá politiku π , ktorá maximalizuje očakávanú návratnosť

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad (20)$$



Obr. 10: Schéma učenia sa s posilňovaním



Obr. 11: Klasifikácia algoritmov RL
[26]

kde $\gamma \in (0, 1]$ je diskontný faktor určujúci význam budúcich odmien.

Metódy RL možno klasifikovať podľa viacerých kritérií. Na obrázku nižšie je znázorненá klasifikácia algoritmov RL, ktorá ich rozdeľuje podľa princípu fungovania[25]:

- **Metódy založené na hodnotách (z angl. *value-based*)**, ktoré aproximujú hodnotovú funkciu stavu $V(s)$ alebo hodnotu akcie $Q(s, a)$, pričom optimálna politika je odvodená implicitne. Typickými predstaviteľmi sú algoritmy Q-learning a Deep Q-Network (DQN).
- **Metódy založené na politikách (z angl. *policy-based*)**, ktoré optimalizujú politiku priamo prostredníctvom gradientného zostupu. Sem patria napríklad REINFORCE alebo algoritmy založené na stochastických politikách.

- **Hybridné metódy (z angl. *actor-critic*)**, ktoré kombinujú oba prístupy. Actor riadi politiku a critic odhaduje hodnotovú funkciu. K známym predstaviteľom patria A2C, A3C alebo Proximal Policy Optimization (PPO).

V modernom výskume RL nadobúda význam najmä pri problémoch s vysokodimenzi-onálnymi stavovými priestormi, ktoré sú riešené prostredníctvom hlbokých neurónových sietí. Tieto tzv. hlboké RL metódy rozšírili použiteľnosť RL aj na zložitejšie úlohy riadenia a rozhodovania.

Použitie bezmodelových (z angl. *model-free*) algoritmov pre 7-DOF manipulátory je podmienené ich schopnosťou efektívne riešiť kinematickú nadbytočnosť a zložité kontaktné interakcie bez vytvárania vysoko presných analytických modelov dynamiky. Tento prístup minimalizuje výpočtové náklady na plánovanie vo vysokorozmerných stavových priestoroch a zabezpečuje vysokú odolnosť voči chybám aproximácie, ktoré sú charakteristické pre zložité viacčlánkové systémy.

3.3.1 Základné princípy RL

Základná štruktúra učenia sa s posilňovaním je založená na interakcii medzi **agentom** a **prostredím**[25]. Agent je entita vykonávajúca rozhodnutia, zatiaľ čo prostredie predstavuje dynamický systém, ktorý na tieto rozhodnutia reaguje.

Stav $s_t \in S$ opisuje aktuálnu konfiguráciu prostredia v čase t . Agent na základe pozorovaného stavu volí **akciu** $a_t \in A$, ktorá ovplyvní budúci vývoj systému. Po vykonaní akcie dostane agent **odmenu** R_{t+1} , ktorá kvantifikuje kvalitu jeho rozhodnutia vzhľadom na cieľ. Prostredie následne prejde do nového stavu podľa prechodovej funkcie $P(s_{t+1} \mid s_t, a_t)$.

Politika $\pi(a \mid s)$ je stratégia, ktorá špecifikuje pravdepodobnosť výberu akcie v danom stave. Politika môže byť deterministická alebo stochastická. Cieľom RL je nájsť optimálnu politiku π^* , ktorá maximalizuje očakávanú budúcu návratnosť.

Celý proces možno zobrazit ako iteratívny cyklus interakcií, v ktorom agent zlepšuje svoju politiku prostredníctvom skúseností. Tento cyklus je kľúčovým mechanizmom adaptívneho správania, ktoré je základom autonómnych riadiacich systémov, robotických aplikácií a ďalších inteligentných technológií.

3.3.2 Algoritmy RL

Ako už bolo spomenuté, na riešenie úlohy plánovania trajektórie 7-DOF robota budú použité moderné model-free algoritmy z oblasti RL. Medzi nimi budú použité:

- PPO (z angl. *Proximal Policy Optimization*) je algoritmus založený na priamom učení politiky, ktorý obmedzuje veľkosť jednotlivých aktualizácií, aby sa tréning

neodklonil príliš ďaleko od predchádzajúceho správania politiky [27]. Vďaka tomu býva stabilný a prakticky dobre použiteľný aj pri zložitejších úlohách riadenia. Nevýhodou môže byť nižšia dátová efektivita v porovnaní s off-policy metódami.

- SAC (z angl. *Soft Actor-Critic*) využíva entropickú regularizáciu, takže sa politika neučí len maximalizovať odmenu, ale aj zachovávať dostatočnú mieru exploračie [28]. Ide o off-policy actor-critic prístup, ktorý býva dátovo efektívny a často dosahuje dobrú stabilitu učenia. V robotike je zaujímavý najmä tým, že dokáže pomerne dobre pracovať aj v spojitých akčných priestoroch.
- TD3 (z angl. *Twin Delayed Deep Deterministic Policy Gradient*) rozširuje deterministické policy-gradient metódy o dvojicu kritikov a oneskorenú aktualizáciu politiky [29]. Tým sa snaží obmedziť nadhodnocovanie hodnotových odhadov, ktoré môže zhoršovať kvalitu učenia. Algoritmus je preto vhodný najmä tam, kde je potrebné presnejšie riadenie v spojitom priestore akcií.
- TQC (z angl. *Truncated Quantile Critic*) modeluje rozdelenie návratnosti pomocou kvantilov a znižuje nadhodnocovanie tým, že časť najvyšších odhadov pri aktualizácii kritika orezáva [30]. Vychádza zo SAC, no snaží sa stabilizovať učenie presnejším odhadom neistoty návratnosti. Tento prístup môže byť výhodný v úlohách, kde sú odhady hodnotovej funkcie citlivé na šum alebo prudké zmeny odmien.

V experimentoch sa pozornosť sústreďí na porovnanie týchto algoritmov s vybranými klasickými plánovačmi.

3.3.3 Uplatnenie RL v praktických úlohách

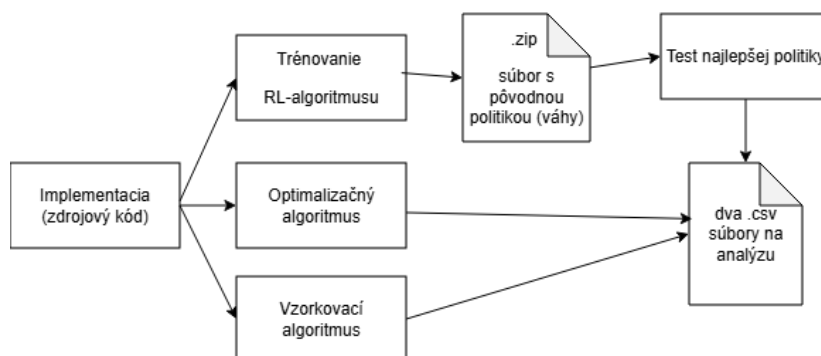
Posilňované učenie nachádza významné uplatnenie v úlohách plánovania pohybu, kde je cieľom autonómneho systému generovať sekvencie akcií vedúce k dosiahnutiu cieľa pri rešpektovaní dynamických obmedzení a bezpečnostných požiadaviek. Medzi najvýznamnejšie oblasti patria autonómna navigácia, vyhýbanie sa prekážkam a ovládanie robotických manipulátorov. Výhodou RL v týchto úlohách je schopnosť učiť sa priamo z interakcie s prostredím bez potreby explicitného modelovania jeho komplexnej dynamiky [25, 31].

V kontexte tejto práce však tieto príklady slúžia predovšetkým ako motivačné pozadie pre výber algoritmov, nie ako samostatne experimentálne overované scenáre. Z praktického hľadiska je preto podstatné najmä to, že RL ponúka možnosť naučiť politiku riadenia priamo zo skúsenosti, hoci za cenu vyšších nárokov na tréning, interpretovateľnosť a prenositeľnosť medzi prostrediami

4 Experimenty

V tejto sekcii sa budeme venovať praktickej implementácii a testovaniu navrhnutej experimentálnej architektúry. Zameriame sa na hardvérové a softvérové aspekty, kritériá hodnotenia, implementáciu a interpretáciu výsledkov. Praktická časť je postavená na troch statických experimentálnych scénach s rozdielnou geometriou prekážok.

Na obrázku je tiež znázornené, ako presne prebiehali experimenty.



Obr. 12: Schéma experimentu

Porovná sa jeden optimalizačný algoritmus, jeden vzorkovací algoritmus a štyri algoritmy NN uvedených vyššie. Pre každý experiment sa ukladajú dva súbory vo formáte CSV: priebehy polôh kĺbov q_1 až q_7 v čase a parametre koncového efektora (ďalej len EE), teda poloha, rýchlosť a zrýchlenie v čase.

4.1 Hardvér a softvér

Na riešenie tejto úlohy bolo použité vývojové prostredie Microsoft Visual Studio Code v operačnom systéme Windows 11. Simulácie boli realizované v jazyku Python 3.11. Boli použité tieto knižnice a nástroje:

- **Gymnasium** - pre simuláciu prostredia a interakciu s robotom[32].
- **Franka Emika Panda** - pre simuláciu a ovládanie robotického ramena[33].
- **PyBullet** - pre fyzikálnu simuláciu a renderovanie prostredia[34].
- **Stable-Baselines3** - pre implementáciu a tréning algoritmov učenia s posilňovaním[31].
- **NumPy** - pre numerické výpočty a manipuláciu s dátami.
- **Matplotlib** - pre vizualizáciu výsledkov a grafov.

- **PyTorch** - pre implementáciu a tréovanie neurónových sietí.
- **Time** - pre meranie času a synchronizáciu vlákien.

Niektoré algoritmy podporovali tréovanie na viacerých vláknach, čo urýchlilo priebeh experimentov. Použitý hardvér pozostával z procesora AMD Ryzen 7 7735, operačnej pamäte Kingston 2×16 GB DDR5 a grafickej karty Nvidia GeForce RTX 4060 Ti.

4.2 Kritériá hodnotenia

Na kvalitné porovnanie trajektórií sú potrebné nielen algoritmy, ktoré sú v daných podmienkach rovnocenné, ale aj zovšeobecnené metriky. Na hodnotenie trajektórií boli zvolené nasledujúce parametre:

- Karteziánska vzdialenosť - predstavuje celkovú dĺžku trajektórie, ktorú prešiel koncový efektor v karteziánskom priestore. Vypočítava sa ako súčet euklidovských vzdialeností medzi po sebe nasledujúcimi polohami koncového efektora zaznamenanými v súboroch .csv.[9]:

$$d_{\text{cart}} = \sum_{i=2}^n \|\mathbf{p}_i - \mathbf{p}_{i-1}\| \quad (21)$$

- Kĺbová vzdialenosť - vzdialenosť medzi aktuálnou konfiguráciou kĺbov a cieľovou konfiguráciou v priestore kĺbov (vypočítava sa na základe logovaných CSV dát q_1 až q_7)[9]:

$$d_{\text{joint}} = \sum_{i=2}^n \sum_{j=1}^m |q_{i,j} - q_{i-1,j}| \quad (22)$$

- Čas trvania - čas potrebný na vykonanie trajektórie od začiatku do konca. V tejto práci ide o praktickú experimentálnu metriku, ktorá nezachytáva len vlastnosti samotného plánovača, ale aj spôsob vykonania trajektórie v simulovanom prostredí.
- Počet lokálnych extrémov v zrýchlení - nepriamy ukazovateľ plynulosti trajektórie. Vyšší počet lokálnych extrémov v priebehoch zrýchlenia znamená viac korekcií a menej plynulý pohyb, zatiaľ čo nižší počet zvyčajne zodpovedá pokojnejšiemu priebehu trajektórie.
- Počet úspešných spustení v rámci jednej politiky - pomer úspešných prípadov, v ktorých robot dosiahol cieľ, k celkovému počtu prípadov. Meralo sa na základe sto spustení každej z úspešných stratégií.

$$S_i = \frac{N_i^{\text{succ}}}{N_i^{\text{all}}}, \quad (23)$$

kde S_i je úspešnosť spustení politiky i , N_i^{succ} je počet úspešných spustení a N_i^{all} je celkový počet spustení danej politiky.

- Priemerný čas potrebný na dosiahnutie cieľa - priemerný čas, ktorý robot potreboval na dosiahnutie cieľa v úspešných prípadoch.

$$T_i = \frac{1}{N_i^{\text{succ}}} \sum_{k=1}^{N_i^{\text{succ}}} t_{i,k}, \quad (24)$$

kde T_i je priemerný čas dosiahnutia cieľa pre politiku i a $t_{i,k}$ je čas potrebný na dosiahnutie cieľa pri k -tom úspešnom spustení.

- Úspešnosť učenia – pomer úspešných politík (tých, ktoré dosiahli úplnú úspešnosť pri učení) vytrénovaných od začiatku k celkovému počtu tréovaných sietí. Tréovali sa štyri siete na troch prípadoch, päť rôznych náhodných inicializáciách (z angl. *seed*).

$$R_i = \frac{N_i^{\text{full}}}{N_i^{\text{train}}}, \quad (25)$$

kde R_i je úspešnosť učenia algoritmu i , N_i^{full} je počet politík, ktoré dosiahli úplnú úspešnosť pri učení, a N_i^{train} je celkový počet tréovaných politík daného algoritmu.

- Rýchlosť učenia – ukazovateľ toho, ako rýchlo sieť dosiahla maximálnu úspešnosť v rámci jedného tréningu. Meria sa ako priemerný počet krokov tréningu za všetky spustenia. Ak sieť nebola dostatočne vytrénovaná, započítal sa maximálny počet krokov.

$$L_i = 1 - \frac{1}{N_i^{\text{train}}} \sum_{j=1}^{N_i^{\text{train}}} n_{i,j}, \quad (26)$$

kde L_i je rýchlosť učenia algoritmu i a $n_{i,j}$ je počet tréningových krokov potrebných na dosiahnutie maximálnej úspešnosti pri j -tom tréningu. Ak politika maximálnu úspešnosť nedosiahla, použije sa hodnota $n_{i,j} = n_{\text{max}}$ (150 000).

V takom prípade sa prvé štyri metriky použijú v porovnávacom experimente, v ktorom sa budú porovnávať riešenia RL s inými riešeniami. Ďalšie štyri parametre (S_i , T_i , R_i , L_i) sa použijú na vzájomné porovnanie riešení RL.

4.3 Implementácia

V tejto podkapitole sa budeme zaoberať samotným vykonaním experimentu – tým, ako boli algoritmy implementované a ako boli hodnotené podľa kritérií opísaných v predchádzajúcej podkapitole.

4.3.1 Definícia robota a úlohy

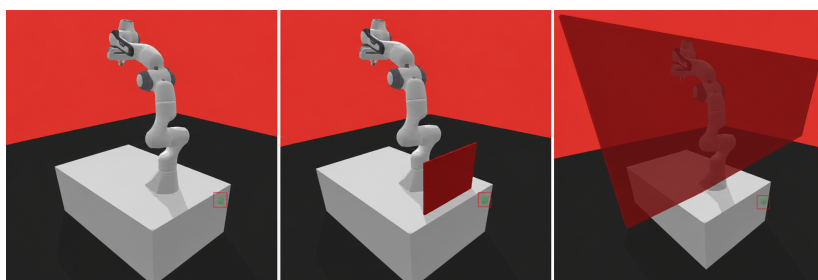
Keďže tieto knižnice využívajú princípy objektovo orientovaného programovania (ďalej len OOP), prostredie umožňuje ľahko definovať úlohu ako triedu:

```
class PandaReachTask(Reach):  
    inicializácia (dedičstvo od hlavnej úlohy Reach)  
    definovanie steny a jej parametrov (farba, rozmery, pozícia)  
    vytvorenie steny v simulácii
```

Podrobný kód sa nachádza v prílohe A. Úloha dosiahnutia[33] (z angl. "reach") bola zvolená kvôli jednoduchosti implementácie a zároveň preto, že prirodzene reprezentuje základnú úlohu presunu koncového efektora do cieľovej polohy.

Keďže model robota sa načíta z knižnice panda, podľa obrázku 12 nám zostáva už len naprogramovať algoritmus. Pokiaľ ide o vzorkovacie alebo optimalizačné algoritmy[1][8], tam sa používajú príslušné matematické metódy; programovanie algoritmu na tréning neurónovej siete je uvedené v nasledujúcej podkapitole.

Prostredie po renderovaní pre experimenty, ktoré budeme vykonávať, vyzerá tak, ako je znázornené na obrázku 13 (prípady 0, 1 a 2 zľava doprava).



Obr. 13: Simulované prostredie pre experimenty

4.3.2 Trénovanie a testovanie RL algoritmov

V podstate sa používajú metódy inicializácie pohybu robota v kĺbovom priestore a definícia modelu neurónovej siete:

```
class JointControlPanda(Panda):
    inicializácia (dedičstvo od hlavnej triedy Panda)
    reset (reset robota do počiatočnej pozície medzi epizódami)

def build_model(algo_name: str, env, seed: int):
    if algo_name == "PPO":
        return PPO(...
                    príslušné hyperparametre
                    ...)
    elif / else s ostatnými algoritmami (SAC, TD3, TQC)
    return model
```

V kóde, ktorý riadi načítanie optimálnej politiky, pracujeme prevažne so súbormi. Po načítaní súboru ZIP s optimálnou politikou a váhami sa po spracovaní a simulácii zapíšu polohy EE a polohy kĺbov 1–7 do súborov CSV.

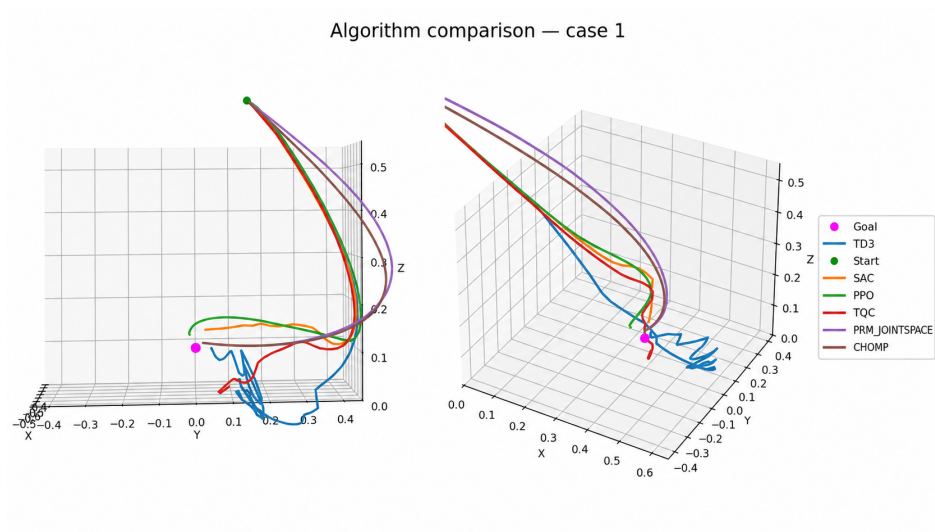
Hlavnými parametrami pri trénovaní sietí sú náhodná inicializácia (*seed*), rýchlosť učenia a veľkosť dávky (*batch size*). SAC využíva entropickú regularizáciu, TD3 pracuje s dvojicou kritikov a oneskorenou aktualizáciou politiky a TQC rozširuje kritiku o kvantilovú reprezentáciu návratnosti[27–30].

4.4 Hodnotenie porovnávacích experimentov

V tejto podkapitole budeme hodnotiť algoritmy a trajektórie, ktoré generujú, na základe kritérií opísaných vyššie. Pre každý z algoritmov bol vybraný jeden optimalizačný algoritmus, jeden vzorkovací algoritmus a štyri algoritmy založené na neurónových sieťach (SAC, PPO, TD3, TQC).

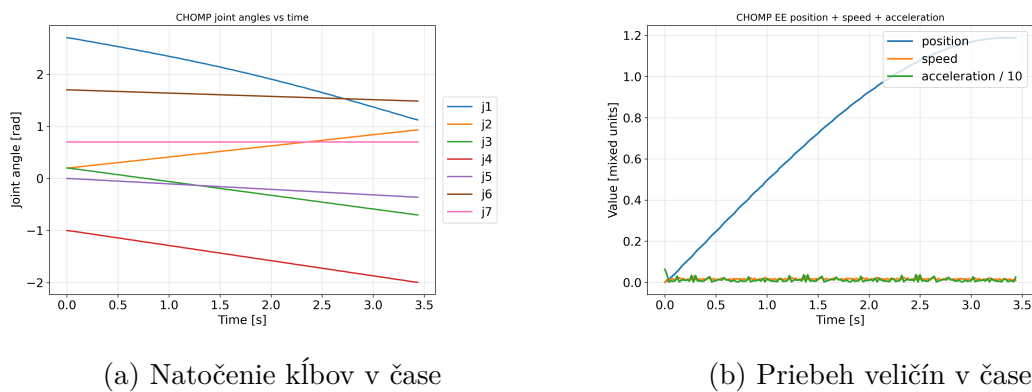
V implementácii bol experiment č. 1 označený ako *case 1*, druhý ako *case 2* a tretí ako *case 3*. V texte však budú nasledujúce grafy, obrázky a tabuľky označené jednotne ako Experiment 1, 2 a 3. V grafoch pohybových parametrov koncového efektora veličina *motion* predstavuje premiestnenie v metroch [m], veličina *speed* rýchlosť v metroch za sekundu [m/s] a veličina *acceleration/10* zrýchlenie, ktoré bolo pre potreby lepšej čitateľnosti grafického zobrazenia zmenšené desaťnásobne [m/s²]. Takéto škálovanie nemení charakter priebehu, ale iba umožňuje prehľadnejšie spoločné zobrazenie viacerých veličín v jednom grafe.

4.4.1 Experiment č. 1: voľný priestor



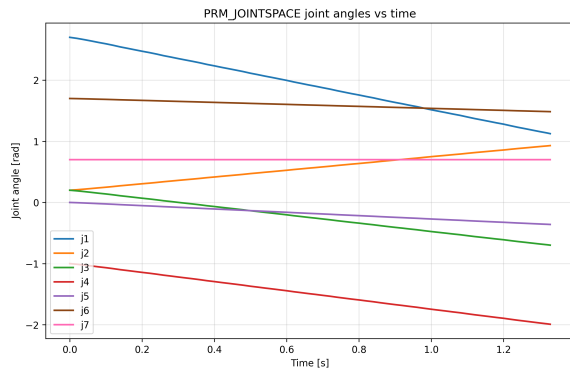
Obr. 14: Experiment 1: Porovnanie výsledkov všetkých algoritmov

Takto vyzerajú trajektórie v prípade, že nie sú žiadne prekážky. Na obrázku 14 je vidieť spoločné porovnanie priebehu pohybu všetkých algoritmov v tomto jednoduchom prostredí.

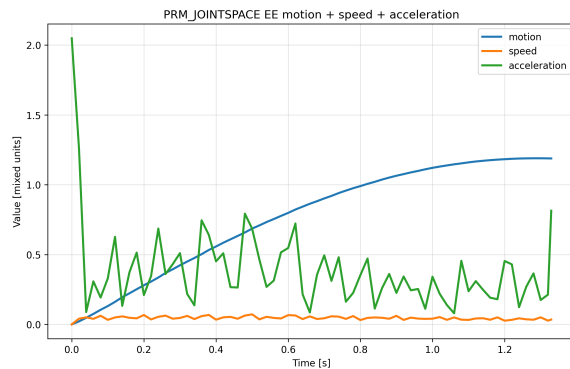


Obr. 15: Experiment 1: Priebeh parametrov v čase pri algoritme CHOMP

Pri algoritme CHOMP možno pozorovať pokojný priebeh pohybu a pomerne vyvážené zmeny kĺbových aj karteziánskych parametrov.



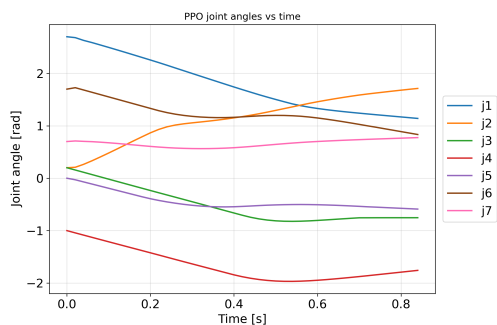
(a) Natočenie kĺbov v čase



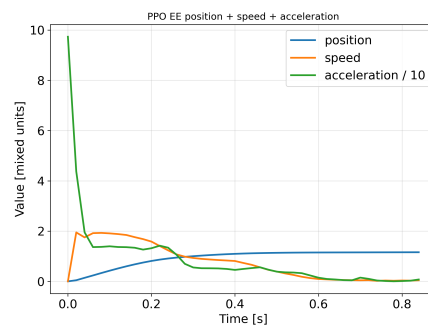
(b) Priebeh veličín v čase

Obr. 16: Experiment 1: Priebeh parametrov v čase pri algoritme PRM

Pri PRM sa oproti CHOMP prejavuje menej priamočiary priebeh, hoci samotné dosiahnutie cieľa zostáva bez výraznejšieho problému.



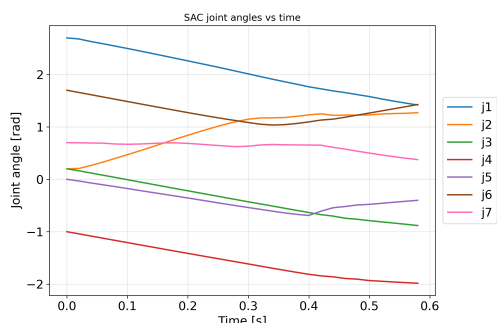
(a) Natočenie kĺbov v čase



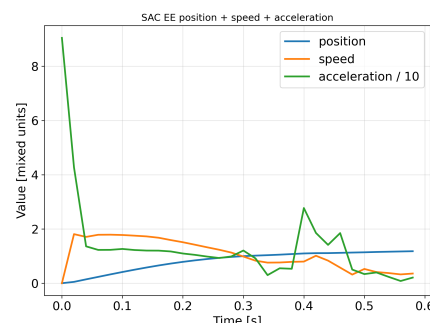
(b) Priebeh veličín v čase

Obr. 17: Experiment 1: Priebeh parametrov v čase pri algoritme PPO

PPO v tomto experimente vykazuje rýchly presun ku cieľu, avšak v závere trajektórie sa objavujú citelnejšie korekcie pohybu.



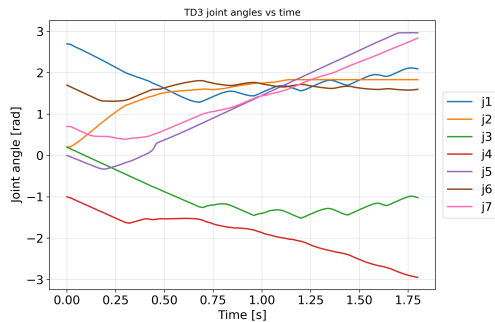
(a) Natočenie kĺbov v čase



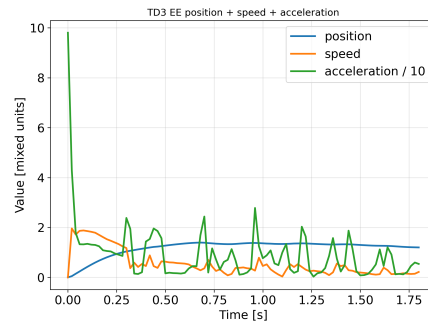
(b) Priebeh veličín v čase

Obr. 18: Experiment 1: Priebeh parametrov v čase pri algoritme SAC

Pri SAC je viditeľný pomerne priamy priebeh trajektórie a priaznivý kompromis medzi rýchlosťou a stabilitou pohybu.



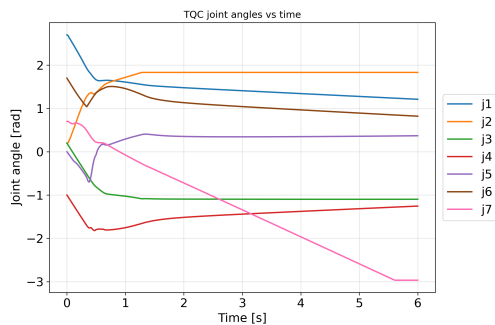
(a) Natočenie kĺbov v čase



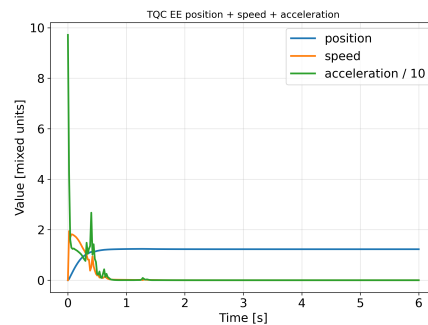
(b) Priebeh veličín v čase

Obr. 19: Experiment 1: Priebeh parametrov v čase pri algoritme TD3

TD3 síce dosahuje cieľ, no v priebehoch kĺbov aj zrýchlenia možno pozorovať väčšiu variabilitu než pri SAC alebo CHOMP.



(a) Natočenie kĺbov v čase



(b) Priebeh veličín v čase

Obr. 20: Experiment 1: Priebeh parametrov v čase pri algoritme TQC

Pri TQC je najvýraznejšie viditeľné oscilovanie v závere pohybu, čo sa následne prejavuje aj v horších trajektórnych metrikách.

Tabuľka 1: Porovnanie algoritmov pre experiment 1

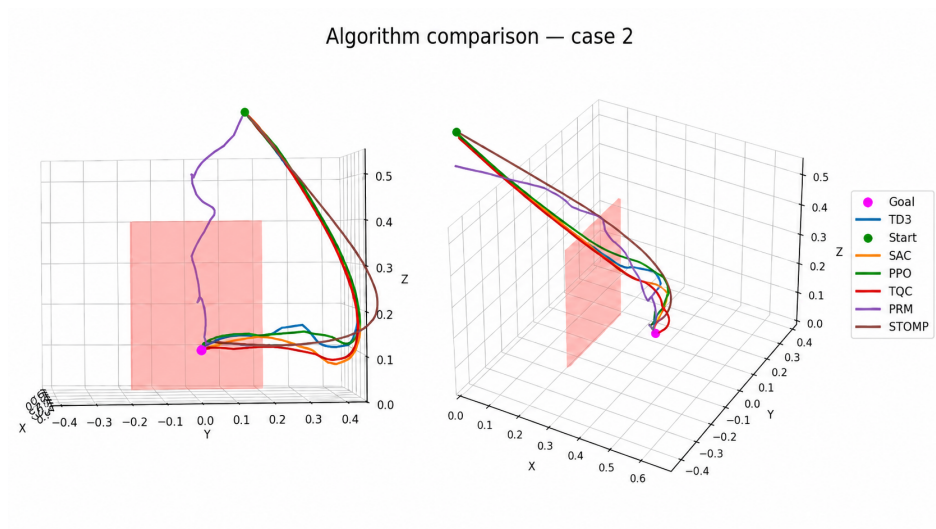
Algoritmus	Karteziánska vzdialenosť [m]	Kľbová vzdialenosť [rad]	Čas [s]
CHOMP	1.753663	4.778970	3.487
PRM	1.765544	4.769384	1.312
PPO	1.761012	7.397882	0.842
SAC	1.679183	6.976589	0.582
TD3	3.081888	18.101776	1.880
TQC	1.730924	13.415086	6.000

V prvom experimente sa zároveň ukazuje, že vo voľnom priestore nevzniká hlavný rozdiel medzi algoritmami v schopnosti nájsť priechod, ale skôr v kvalite samotného pohybu. Na obrázku 14 a v detailných priebehoch jednotlivých algoritmov možno pozorovať, že algoritmy SAC a PPO volia pomerne priamy presun ku cieľu, zatiaľ čo TD3 a najmä TQC vykazujú menej pokojný záver trajektórie. Podobný trend je viditeľný aj v priebehoch zrýchlenia, kde pokojnejší pohyb zodpovedá menšiemu počtu lokálnych extrémov a teda menšiemu počtu dodatočných korekcií. Tento prípad preto dobre ukazuje, že aj pri jednoduchej úlohe nestačí hodnotiť riešenie iba podľa času vykonania, ale aj podľa plynulosti a predvídateľnosti pohybu.

Dôvod je v tom, že voľný priestor nevyžaduje od algoritmu zložitú rozhodovanie o obchádzaní prekážky, takže sa naplno prejaví samotný princíp generovania trajektórie. CHOMP optimalizuje celú trajektóriu naraz a prirodzene penalizuje prudšie zmeny, preto jeho priebehy pôsobia najspojitejšie. Naopak RL politiky reagujú sekvenčne krok po kroku; ak sa v závere pohybu snažia ešte spresniť polohu koncového efektora, prejaví sa to drobnými dodatočnými zásahmi do riadenia, ktoré sa v grafoch rýchlosti a najmä zrýchlenia zobrazia ako lokálne extrémny. Preto sú rozdiely medzi SAC, PPO, TD3 a TQC v tomto experimente skôr rozdielmi v kultivovanosti finálneho priblíženia než v samotnej schopnosti cieľ dosiahnuť.

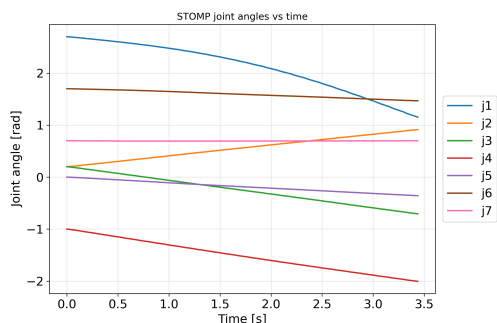
Z grafov a tabuľky je zrejmé, že najrýchlejší bol algoritmus SAC (0.582 sekundy), avšak najplynulejšiu a najoptimálnejšiu trajektóriu mal práve CHOMP (najlepšia kombinácia kartézskej a kľbovej vzdialenosti), čo však bolo podmienené dlhším časom spracovania. Najdlhší a zároveň neoptimálny by bol TQC – v blízkosti cieľa začal výrazne oscilovať, čo je vidieť na obrázkoch 20b a 20a.

4.4.2 Experiment č. 2: malá stena

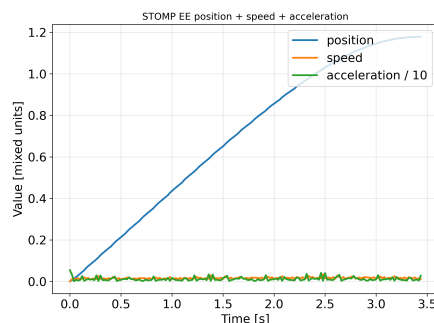


Obr. 21: Experiment 2: Porovnanie výsledkov všetkých algoritmov

V druhom experimente už prekážka núti algoritmy voliť zreteľnejšie odlišné stratégie obchádzania priestoru.



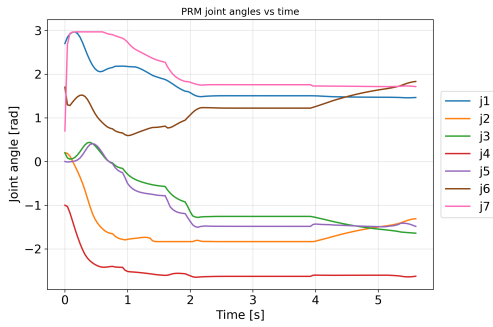
(a) Natočenie kĺbov v čase



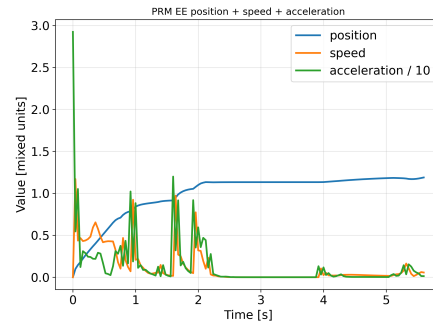
(b) Priebeh veličín v čase

Obr. 22: Experiment 2: Priebeh parametrov v čase pri algoritme STOMP

Pri STOMP je viditeľný plynulý priebeh pohybu, ktorý dobre zodpovedá jeho priaznivým výsledkom v trajektórnych metrikách.



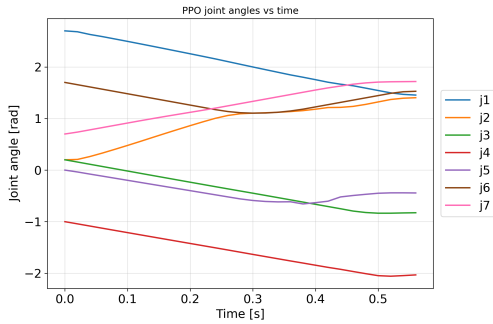
(a) Natočenie kĺbov v čase



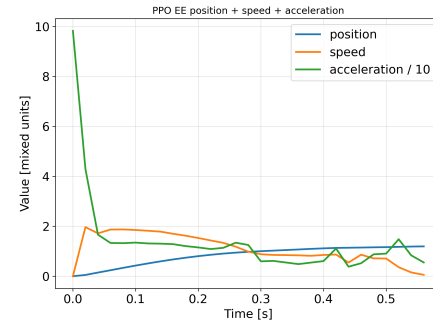
(b) Priebeh veličín v čase

Obr. 23: Experiment 2: Priebeh parametrov v čase pri algoritme PRM

PRM v tomto prípade ukazuje, že priaznivá karteziánska dráha ešte nemusí znamenať úsporný pohyb v priestore kĺbov.



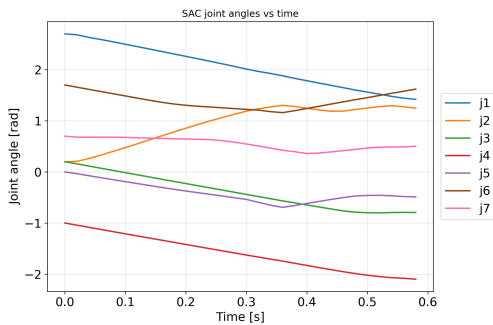
(a) Natočenie kĺbov v čase



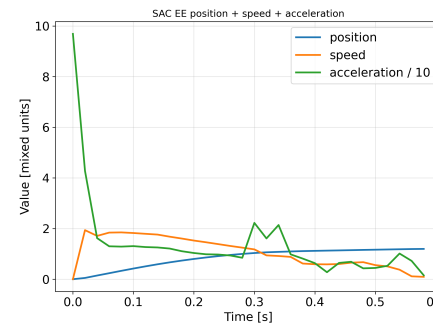
(b) Priebeh veličín v čase

Obr. 24: Experiment 2: Priebeh parametrov v čase pri algoritme PPO

PPO opäť pôsobí rýchlo, no v závere pohybu je možné sledovať mierne korekcie súvisiace s obchádzaním prekážky.



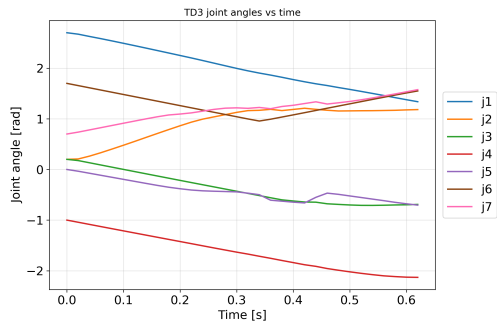
(a) Natočenie kĺbov v čase



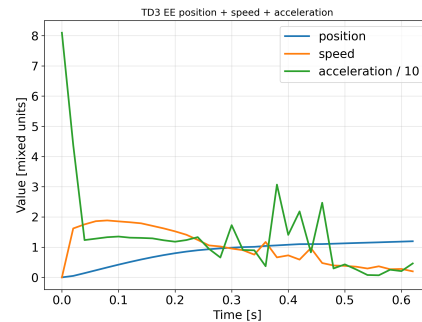
(b) Priebeh veličín v čase

Obr. 25: Experiment 2: Priebeh parametrov v čase pri algoritme SAC

SAC si zachováva pomerne stabilný profil pohybu aj pri prítomnosti malej steny, čo podporuje jeho vyvážené hodnotenie.



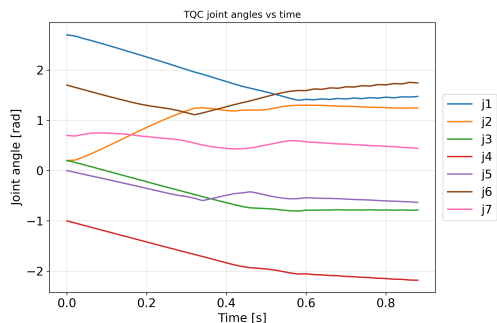
(a) Natočenie kĺbov v čase



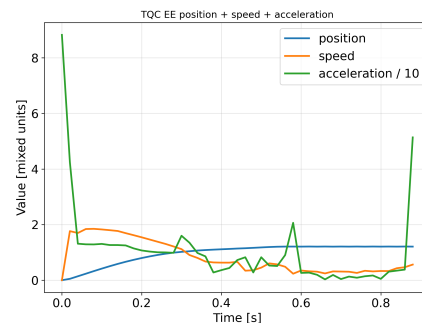
(b) Priebeh veličín v čase

Obr. 26: Experiment 2: Priebeh parametrov v čase pri algoritme TD3

Pri TD3 je stále prítomná vyššia variabilita priebehov, hoci samotné dosiahnutie cieľa ostáva spoľahlivé.



(a) Natočenie kĺbov v čase



(b) Priebeh veličín v čase

Obr. 27: Experiment 2: Priebeh parametrov v čase pri algoritme TQC

TQC aj v tomto experimente potvrdzuje rýchly pohyb, no za cenu mierne menej pokojného záveru trajektórie.

Tabuľka 2: Porovnanie algoritmov pre experiment 2

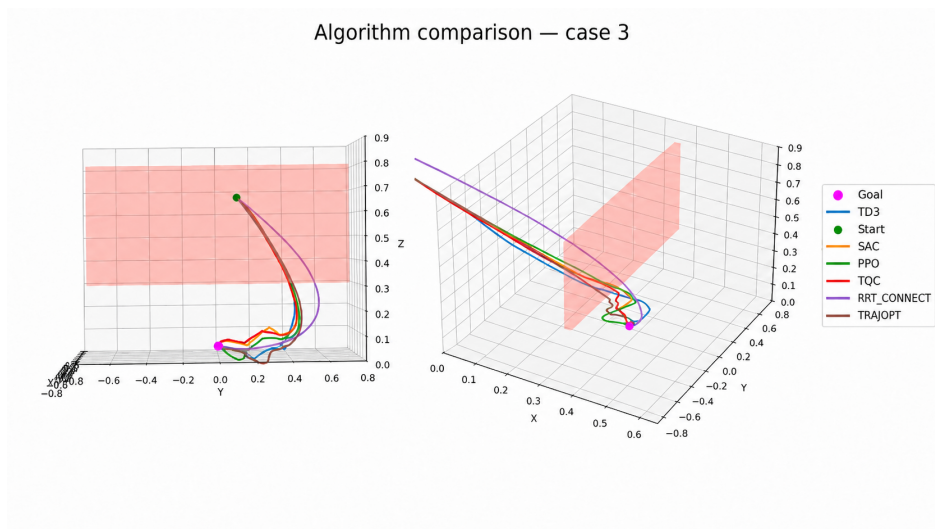
Algoritmus	Karteziánska vzdialenosť [m]	Kľbová vzdialenosť [rad]	Čas [s]
STOMP	1.715463	4.776812	3.452
PRM	1.493292	18.512422	5.687
PPO	1.717286	7.492869	0.566
SAC	1.723389	7.214949	0.584
TD3	1.707741	8.023564	0.612
TQC	1.828552	8.171624	0.855

Pri druhom experimente sa už viac prejavuje schopnosť algoritmu reagovať na priestorové obmedzenie a viesť trajektóriu s ohľadom na prekážku. STOMP si v tabuľke 2 zachováva vyvážený výsledok z hľadiska plynulosti, pričom jeho priebeh na obrázku 21 a v detaile 22 pôsobí konzistentnejšie než pri väčšine ostatných prístupov. Aj pri tejto úlohe možno z grafov usudzovať, že menší počet lokálnych extrémov v zrýchlení zodpovedá pokojnejšiemu obchádzaniu steny, zatiaľ čo častejšie extrémy naznačujú viac priebežných korekcií. Zároveň sa ukazuje, že priaznivá karteziánska vzdialenosť ešte nemusí znamenať kvalitnú trajektóriu celého manipulátora, ak je dosiahnutá za cenu veľkých zmien v priestore kĺbov.

Malá stena núti manipulátor zmeniť konfiguráciu skôr, než sa koncový efektor dostane do blízkosti cieľa, a práve preto sa rozdiely medzi algoritmami prenášajú aj do priebehov kĺbov. STOMP pracuje s perturbáciami celej trajektórie a následným vyhladením, takže vie prekážku obísť bez výrazných lokálnych zásahov v závere pohybu. PRM síce nájde pomerne krátku dráhu koncového efektora, no keďže rozhoduje v diskrétnom vzorkovanom priestore konfigurácií, môže medzi jednotlivými uzlami zvoliť menej hospodárnu zmenu držania ramena; práve preto má priaznivú karteziánsku vzdialenosť, ale veľmi nevýhodnú kľbovú vzdialenosť. Pri RL algoritmoch sa zase prejavuje, že obchádzanie steny prebieha úspešne, no finálne doladovanie smeru pohybu vytvára viac korekcií než pri najpokojnejšom klasickom riešení.

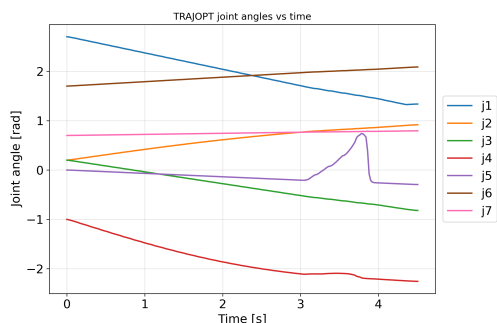
Z grafov a tabuľky je zrejmé, že v tomto prípade bol najrýchlejší algoritmus PPO (0.566 sekundy), avšak najplynulejšiu a najoptimálnejšiu trajektóriu mal algoritmus STOMP (najlepšia kombinácia kartézskej a kľbovej vzdialenosti), čo však bolo podmienené dlhším časom spracovania. Na druhej strane je na obrázku 21 vidieť, že každý z algoritmov založených na neurónových sieťach vykonal pohyb po kratšej oblúkovej dráhe, hoci na konci mierne osciloval. Najdlhší a zároveň najmenej hospodárny bol PRM, pretože pri pohybe v blízkosti steny volil menej opatrné zmeny konfigurácie, čo je vidieť aj na obrázku 23a s priebehom natočenia kĺbov 1 až 7 v čase.

4.4.3 Experiment č. 3: veľká závesná stena

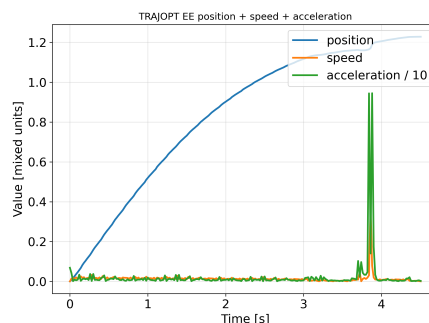


Obr. 28: Experiment 3: Porovnanie výsledkov všetkých algoritmov

Tretí experiment už kladie najväčší dôraz na presné vedenie trajektórie v obmedzenom priestore.



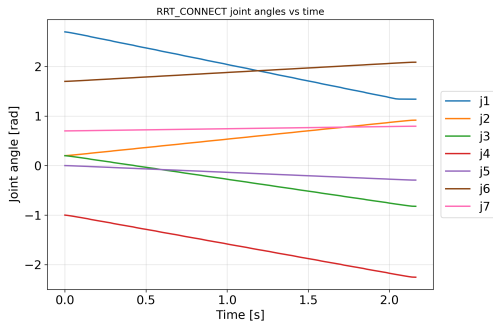
(a) Natočenie kĺbov v čase



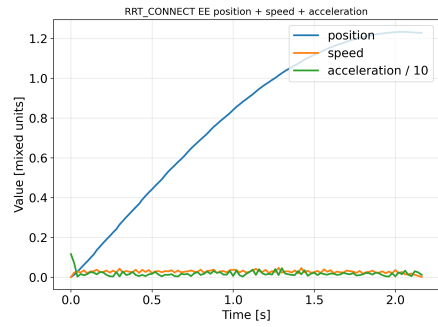
(b) Priebeh veličín v čase

Obr. 29: Experiment 3: Priebeh parametrov v čase pri algoritme TrajOpt

Pri TrajOpt je viditeľná snaha o relatívne kompaktný pohyb, no v tomto prostredí sa zároveň prejavuje citlivosť na kolízne vedenie trajektórie.



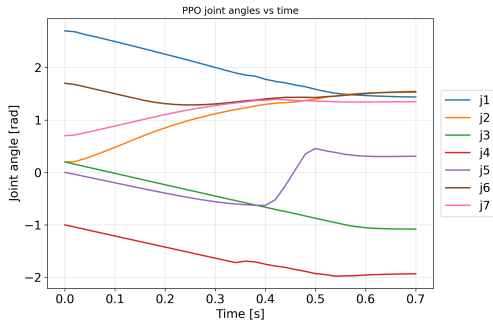
(a) Natočenie kĺbov v čase



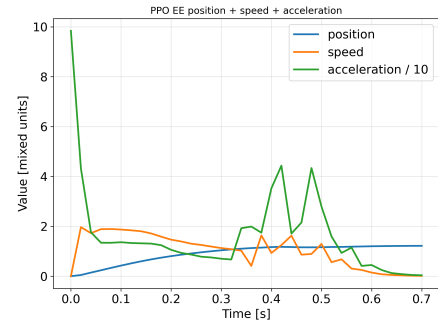
(b) Priebeh veličín v čase

Obr. 30: Experiment 3: Priebeh parametrov v čase pri algoritme RRT-Connect

RRT-Connect v tomto prípade pôsobí veľmi presvedčivo najmä z pohľadu kĺbovej úspornosti a pokojnejšieho vedenia pohybu.



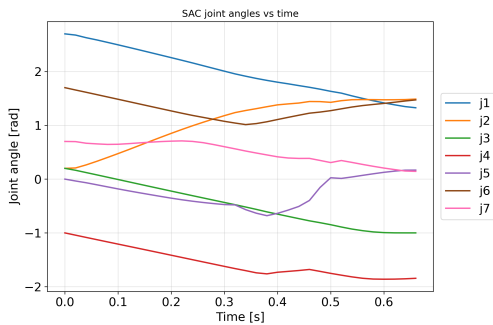
(a) Natočenie kĺbov v čase



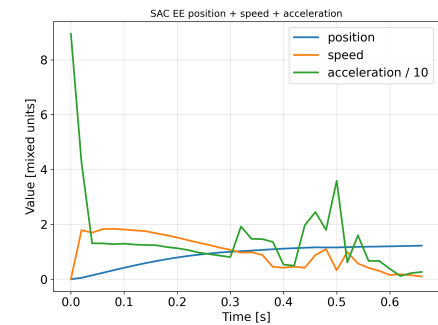
(b) Priebeh veličín v čase

Obr. 31: Experiment 3: Priebeh parametrov v čase pri algoritme PPO

PPO si zachováva rýchlosť, no v detailných priebehoch je stále badateľná väčšia potreba korekcií v závere trajektórie.



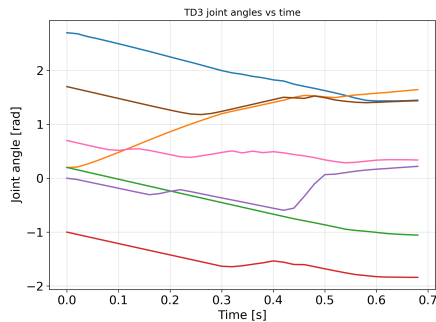
(a) Natočenie kĺbov v čase



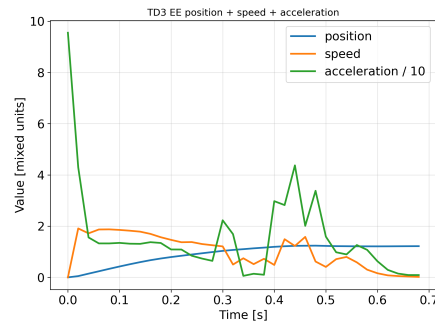
(b) Priebeh veličín v čase

Obr. 32: Experiment 3: Priebeh parametrov v čase pri algoritme SAC

SAC aj v treťom experimente ponúka pomerne vyvážený profil, hoci už menej výrazne než v jednoduchších scénach.



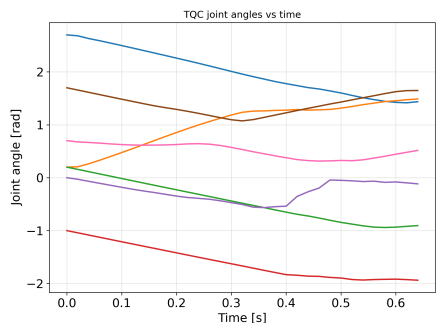
(a) Natočenie kĺbov v čase



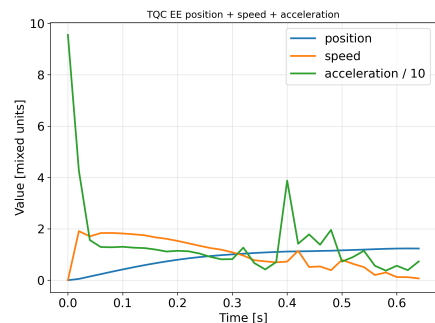
(b) Priebeh veličín v čase

Obr. 33: Experiment 3: Priebeh parametrov v čase pri algoritme TD3

Pri TD3 sú znovu viditeľné väčšie zmeny kĺbových parametrov, ktoré znižujú celkovú plynulosť pohybu.



(a) Natočenie kĺbov v čase



(b) Priebeh veličín v čase

Obr. 34: Experiment 3: Priebeh parametrov v čase pri algoritme TQC

TQC dosahuje rýchly pohyb, ale podobne ako v predchádzajúcich experimentoch za cenu menej pokojného záverečného správania.

Tabuľka 3: Porovnanie algoritmov pre experiment 3

Algoritmus	Karteziánska vzdialenosť [m]	Kĺbová vzdialenosť [rad]	Čas [s]
TRAJOPT	1.778515	7.095145	4.621
RRT-C	1.829370	5.127783	2.161
PPO	1.886781	8.251197	0.700
SAC	1.754269	8.404129	0.677
TD3	1.843097	8.650320	0.689
TQC	1.733804	7.805484	0.645

Tretí experiment predstavuje najnáročnejšie prostredie, pretože veľká závesná stena núti algoritmy voliť presnejší a mechanicky rozumnejší manéver. Z tabuľky 3 vyplýva, že RRT-Connect dosiahol najnižšiu kĺbovú vzdialenosť, čo sa potvrdzuje aj na obrázku 28 a v detaile 30, kde jeho trajektória pôsobí kompaktnejšie a vyrovnanejšie. V priebehoch zrýchlenia sa zároveň ukazuje, že pokojnejší pohyb býva sprevádzaný nižším počtom lokálnych extrémov, zatiaľ čo výraznejšie oscilácie na konci trajektórie tento počet zvyšujú. V tomto prípade sa preto ešte výraznejšie ukazuje rozdiel medzi rýchlym dosiahnutím cieľa a skutočne hladkým vedením celej kinematickej štruktúry robota.

Väčšia prekážka už výraznejšie mení topológiu priechodného priestoru, takže nestačí iba lokálne korigovať smer pohybu, ale treba zvoliť celkovo vhodný manéver celej sústavy. RRT-Connect tu ťaží z toho, že pri hľadaní spojenia dvoch stromov dokáže pomerne rýchlo objaviť priechod, ktorý je síce o niečo dlhší v karteziánskom priestore, ale úspornejší v priestore kĺbov. Naopak TrajOpt je ako lokálna optimalizačná metóda citlivejší na inicializačnú trajektóriu a na blízkosť kolíznych oblastí, takže v zložitejšom prostredí ľahšie vzniknú nežiaduce zmeny konfigurácie. RL politiky si zachovávajú rýchlosť aj pri tejto úlohe, no pri tesnejšom obchádzaní prekážky a pri finálnom dosadaní do cieľa sa častejšie objavujú krátke korekčné zásahy, ktoré zvyšujú počet extrémov v zrýchlení.

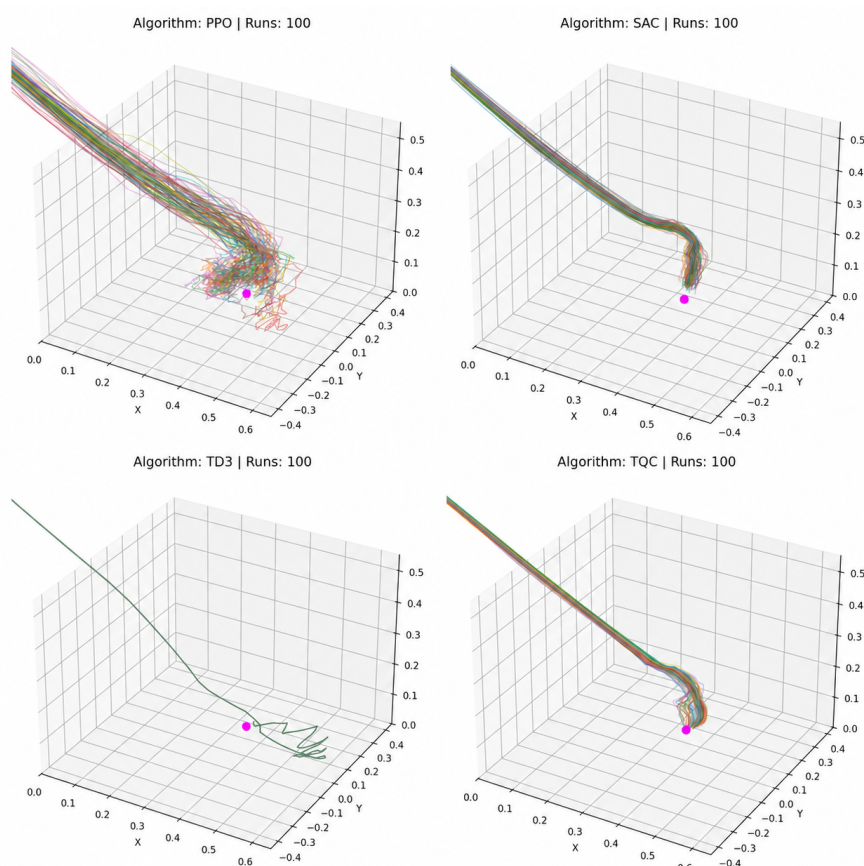
V treťom experimente bol najrýchlejší algoritmus TQC (0.645 sekundy), avšak z hľadiska plynulosti trajektórie pôsobil najpresvedčivejšie algoritmus RRT-Connect (v tabuľke označený ako RRT-C). Tentoraz sa optimalizačný Trajopt chytil jedným z kĺbov o stôl, čo je vidieť na obrázkoch 28 a 29a, čo zhoršilo jeho kĺbovú vzdialenosť pred RRT-Connectom, ktorý sa vyhol stolu. Na obrázku 28 je vidieť, že algoritmy založené na neurónových sieťach sa pohybovali po kratšej oblúkovej dráhe, ale na konci mierne oscillovali, čo tiež zhoršilo ich kĺbovú vzdialenosť.

4.5 Hodnotenie vzajomných RL experimentov

Cieľom tohto experimentu bude kvantitatívne vyhodnotiť správanie samotného algoritmu v rôznych experimentoch. Okrem bežných experimentov, v ktorých sme porovnávali prístupy založené na strojovom učení s klasickými, by sme mali porovnať aj robustnosť a stabilitu. Vtedy boli popísané štyri parametre, uvedené v časti o kritériách hodnotenia, ktoré sa týkajú algoritmov RL: úspešnosť S_i , priemerný čas T_i , úspešnosť učenia R_i a rýchlosť učenia L_i .

V tejto podkapitole možno robustnosť chápať ako schopnosť politiky udržiavať porovnateľný výkon pri opakovaných spusteniach a v rôznych prípadoch tej istej úlohy. Adaptabilita sa tu prejavuje tým, do akej miery si jedna naučená politika dokáže zachovať použiteľné správanie aj pri náročnejšom priebehu pohybu alebo pri odlišnom usporiadaní prekážok v rámci porovnávaných experimentov.

4.5.1 Správanie RL v experimente č. 1



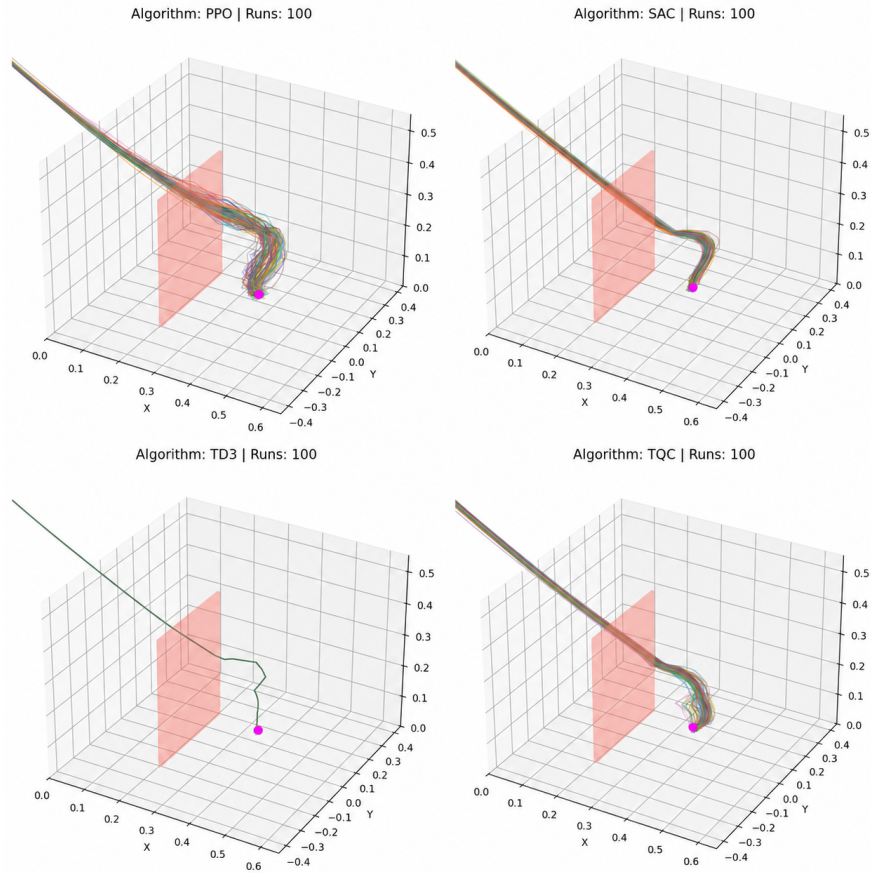
Obr. 35: Rozptyl trajektórie v rámci tej istej politiky, experiment 1

Parameter/algoritmus	PPO	SAC	TD3	TQC
Úspešnosť S_i	0.58	0.88	1	0.98
Priemerný čas T_i	0.874	0.685	2.76	1.23
Úspešnosť učenia R_i	1	1	1	0.8
Rýchlosť učenia L_i	0.622	0.766	0.813	0.826

Tabuľka 4: Porovnanie metrík RL-algoritmov pre experiment 1

V prvom experimente dosiahol najvyššiu úspešnosť algoritmus TD3, pričom SAC a TQC sa k nemu výrazne priblížili. Z tabuľky 4 je zároveň vidieť, že SAC poskytuje najlepší kompromis medzi úspešnosťou, časom a úspešnosťou učenia, kým PPO zaostáva najmä v úspešnosti a TQC napriek dobrej rýchlosti učenia stráca práve v úspešnosti učenia. Pre jednoduché prostredie bez prekážok sa teda javí ako najvyvázenejší práve SAC, zatiaľ čo TD3 pôsobí ako najspoľahlivejší z pohľadu samotného dokončenia úlohy.

4.5.2 Správanie RL v experimente č. 2



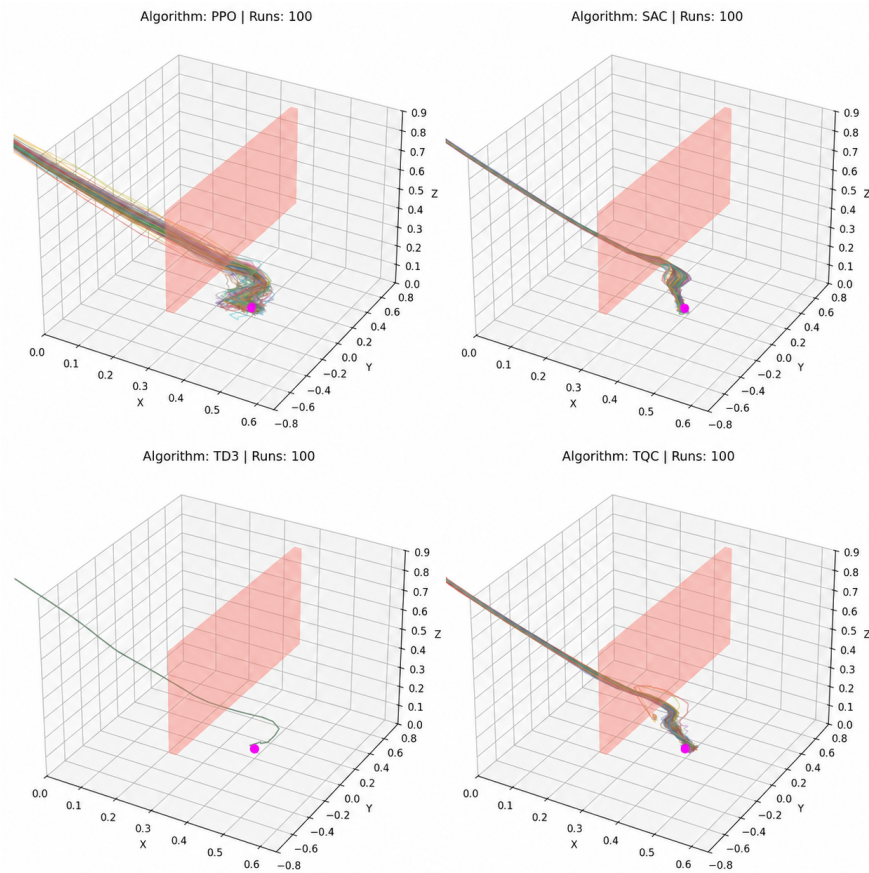
Obr. 36: Rozptyl trajektórie v rámci tej istej politiky, experiment 2

Parameter/algoritmus	PPO	SAC	TD3	TQC
Úspešnosť S_i	0.82	0.96	1.00	0.99
Priemerný čas T_i	0.585	0.622	0.617	0.898
Úspešnosť učenia R_i	1	1	1	1
Rýchlosť učenia L_i	0.346	0.681	0.486	0.760

Tabuľka 5: Porovnanie metrík RL-algoritmov pre experiment 2

V druhom experimente sa hodnoty úspešnosti všetkých RL algoritmov zvýšili, čo naznačuje, že prítomnosť malej steny nepredstavovala pre naučené politiky zásadný problém. TD3 opäť dosiahol úplnú úspešnosť a SAC s TQC si udržali veľmi silné výsledky, pričom PPO bol najrýchlejší z hľadiska priemerného času podľa tabuľky 5. Z hľadiska celkového profilu však znovu pôsobí presvedčivo SAC, pretože kombinuje vysokú úspešnosť, plnú úspešnosť učenia a priaznivý čas bez výraznejšieho kompromisu v niektorej metrike.

4.5.3 Správanie RL v experimente č. 3



Obr. 37: Rozptyl trajektórie v rámci tej istej politiky, experiment 3

Parameter/algorithmus	PPO	SAC	TD3	TQC
Úspešnosť S_i	0.88	0.97	1	0.92
Priemerný čas T_i	0.702	0.692	0.711	0.688
Úspešnosť učenia R_i	0.6	1	1	0.8
Rýchlosť učenia L_i	0.378	0.675	0.665	0.788

Tabuľka 6: Porovnanie metrík RL-algoritmov pre experiment 3

V treťom experimente sa rozdiely medzi algoritmami prejavili najmä v úspešnosti učenia a v schopnosti udržať výkon v zložitejšom priestore. SAC a TD3 si zachovali plnú úspešnosť učenia, pričom TD3 dosiahol najvyššiu úspešnosť a TQC bol najrýchlejší aj s najvyššou rýchlosťou učenia podľa tabuľky 6. Napriek tomu sa ako najvyváženejší javí SAC, pretože si udržuje veľmi vysokú úspešnosť a zároveň neprejavuje pokles úspešnosti učenia, ktorý je pri PPO a čiastočne aj pri TQC už zreteľný.

4.6 Výsledky experimentov

Z porovnania výsledkov všetkých troch experimentov je zrejmé, že optimalizačné algoritmy (CHOMP, STOMP, TrajOpt) majú tendenciu generovať hladšie a optimalizovanejšie trajektórie, avšak za cenu dlhšieho času spracovania. Na druhej strane, algoritmy založené na neurónových sieťach (PPO, SAC, TD3, TQC) sú výrazne rýchlejšie, ale ich trajektórie sú často menej optimalizované a môžu vykazovať oscilácie, najmä v zložitejších prostrediach s prekážkami. Vzorkovací algoritmus PRM sa ukázal ako nevhodný pre zložité prostredia, kde je potrebné precízne manévrovanie okolo prekážok, zatiaľ čo RRT-Connect sa osvedčil ako efektívny v generovaní hladších trajektórií v zložitejších prostrediach. Celkovo je zrejmé, že výber algoritmu závisí na konkrétnych požiadavkách úlohy, ako je potreba rýchlosti oproti kvalite trajektórie a zložitosť prostredia.

Na druhej strane, ak posudzujeme dynamické parametre EE, treba tieto výsledky interpretovať opatrne. Niektoré priebehy rýchlosti a zrýchlenia naznačujú, že pri priamom prenose do reálneho robotického systému by bolo potrebné doplniť ďalšie obmedzenia alebo vyhladenie trajektórie. Takéto prudké zmeny možno čiastočne vysvetliť diferenciáciou signálu, zmenou správania agenta počas učenia a mechanizmom riadenia polohy v simulátore. Pri robote Franka Panda navyše nemožno ľubovoľne preniesť akcie zo simulácie priamo na pohony, pretože rozhranie FCI a knižnica *libfranka* predpokladajú rešpektovanie obmedzení rýchlosti, zrýchlenia, trhu (z angl. *jerk*) a pri externom riadení aj obmedzení zmeny momentu [35, 36].

Pri detailnejšom pohľade na jednotlivé experimenty sa ukazuje, že voľný priestor najlepšie odhalil samotný charakter pohybu jednotlivých prístupov. Keď algoritmus

nebol obmedzený prekážkou, rozdiely sa neprejavovali v tom, či sa cieľ podarí dosiahnuť, ale skôr v tom, ako kultivovane sa k nemu robot dostane. Práve preto sa v prvom experimente mohli naplno prejať výhody CHOMP-u z pohľadu plynulosti a zároveň nevýhody niektorých RL politík, pri ktorých sa v závere trajektórie objavovali väčšie korekcie. Tento prípad naznačuje, že už v jednoduchej úlohe má zmysel rozlišovať medzi rýchlym a kvalitným pohybom, pretože tieto dve vlastnosti nemusia automaticky kráčať spolu.

Druhý experiment s malou stenou ukázal, že aj relatívne nenápadná prekážka môže zmeniť spôsob, akým treba výsledky interpretovať. V tejto situácii už nestačí sledovať iba to, či je trajektória krátka, ale aj to, aký pohyb musí robot vykonať v priestore kĺbov, aby sa stene bezpečne vyhol. STOMP sa v tomto prípade ukázal ako priaznivé riešenie z pohľadu hladkosti pohybu, zatiaľ čo PRM síce dosiahol nízku karteziánsku vzdialenosť, ale za cenu veľmi vysokej kĺbovej vzdialenosti a dlhého času vykonania. Takýto výsledok je dôležitý najmä metodicky, pretože potvrdzuje, že priaznivá hodnota jednej metriky ešte sama osebe nevypovedá o praktickej kvalite celej trajektórie.

Najnáročnejší bol tretí experiment, kde veľká závesná stena výraznejšie preverila schopnosť algoritmov viesť pohyb nielen presne, ale aj mechanicky rozumne. V tomto prostredí sa ukázalo, že výhoda rýchlych RL prístupov sa síce zachováva, no s rastúcou zložitou priestoru rastie aj význam trajektórie, ktorá je hladká počas celej sekvencie a nevyžaduje nadmerné zmeny konfigurácie ramena. RRT-Connect preto v tomto experimente pôsobí ako vyváženejšie riešenie než viaceré rýchlejšie prístupy, pretože ponúkol priaznivejší kompromis medzi priechodnosťou priestoru a hospodárnosťou pohybu. Tento výsledok zároveň naznačuje, že pri zložitejších scénach môže byť vhodný klasický plánovací prístup stále konkurencieschopný.

Samostatné hodnotenie RL algoritmov prinieslo doplnkový pohľad, ktorý by sa z bežného porovnania trajektórií nemusel ukázať úplne zreteľne. PPO, SAC, TD3 a TQC sa od seba neodlišujú len výslednou trajektóriou, ale aj úspešnosťou učenia, úspešnosťou pri opakovaných spusteniach a rýchlosťou, s akou sa dostanú k použiteľnej politike. Z troch porovnávaných prípadov vychádza najvyrovnanejšie SAC, pretože si udržiava veľmi vysokú úspešnosť, priaznivý čas aj dobrú úspešnosť učenia naprieč prostrediami. TD3 dosahuje v niektorých prípadoch ešte vyššiu úspešnosť, no z hľadiska celkového profilu nepôsobí vždy rovnako vyvážene, zatiaľ čo TQC zaujme rýchlosťou učenia, ale nie vo všetkých prípadoch si zachová rovnakú úspešnosť učenia.

Z praktického hľadiska je dôležité, že výsledky tejto práce nenaznačujú existenciu jedného univerzálne najlepšieho algoritmu. Skôr ukazujú, že vhodnosť riešenia závisí od toho, či je prioritou rýchla reakcia, plynulosť trajektórie, stabilita správania alebo bezpečné vedenie pohybu v komplikovanejšom priestore. Ak je cieľom čo najrýchlejšia

inferencia po natrénovaní modelu, RL prístupy v daných experimentoch vykazujú výhodu. Ak je naopak dôležité dosiahnuť hladšiu a mechanicky rozumnejšiu trajektóriu, klasické optimalizačné alebo vhodne zvolené vzorkovacie algoritmy sa ukázali ako priaznivé riešenie.

Pri interpretácii časových rozdielov medzi RL a optimalizačnými prístupmi treba zároveň dodať, že nižšia rýchlosť klasických algoritmov je v tejto implementácii aspoň čiastočne ovplyvnená aj spôsobom vykonania trajektórie v simulovanom prostredí. Časový priebeh v zaznamenaných dátach preto neodráža iba vlastnosti samotného plánovača, ale aj následné vykonanie trajektórie a jeho riadiacu vrstvu. Z tohto dôvodu je vhodné brať nižšiu rýchlosť CHOMP, STOMP alebo TrajOpt ako kombináciu vlastností algoritmu aj vlastností vývojového a simulačného prostredia.

Z pohľadu algoritmov RL je zároveň dôležité spomenúť otázku adaptability. Výsledky v tejto podkapitole naznačujú, že jedna naučená politika vie v rámci daného experimentu reagovať na opakované spustenia a zachovať použiteľné správanie, čo možno považovať za prejav určitej adaptability. Na druhej strane to ešte neznamená automatickú prenositeľnosť do nového prostredia, pretože pri výraznejšej zmene geometrie úlohy alebo prekážok je často potrebné sieť znovu dotrénovať alebo natrénovať od začiatku.

Osobitnú pozornosť si zasluhujú aj dynamické parametre koncového efektora. V simulácii je síce možné dosiahnuť veľmi rýchly pohyb a prudké zmeny zrýchlenia, no pri prenose na reálne robotické rameno by takýto priebeh mohol predstavovať problém z pohľadu mechanického zaťaženia, bezpečnosti alebo presnosti vykonania. Preto je vhodné interpretovať úspešnosť RL prístupov opatrne: vysoká rýchlosť a dobré splnenie cieľa ešte automaticky neznamenajú pripravenosť na fyzickú implementáciu. V literatúre sa síce ukazuje, že RL možno na robote Franka Panda nasadiť aj v reálnom prostredí a prepojiť ho s klasickou riadiacou vrstvou [37], no takýto prenos spravidla neprebíha ako jednoduché odoslanie surovej akcie z neurónovej siete na motory. Zvyčajne je potrebná dodatočná bezpečnostná vrstva, filtrovanie príkazov, rešpektovanie hardvérových limitov a často aj sim-to-real postupy alebo doladenie politiky na skutočnom zariadení [38, 39]. V tomto smere práca ukazuje, že pri hodnotení algoritmov plánovania pohybu treba brať do úvahy nielen výsledok, ale aj spôsob, akým bol tento výsledok dosiahnutý.

S tým súvisí aj skutočnosť, že neurónové siete majú v tomto kontexte povahu modelov typu *black box*. Aj keď môžu v experimentoch dosahovať priaznivé výsledky, z výslednej trajektórie nie je vždy jednoznačne zrejmé, prečo sa politika rozhodla práve pre daný spôsob pohybu. Pri prenose do reálneho systému je to dôležité aj z bezpečnostného hľadiska, pretože nie je úplne transparentné, aká vnútorná reprezentácia viedla k konkrétnej akcii a ako sa politika zachová pri menej očakávanej situácii. Práve preto ostáva ich nasadenie do citlivých alebo bezpečnostne náročných úloh do istej

miery rizikové a vyžaduje opatrnejšiu interpretáciu než pri klasických algoritmoch, pri ktorých je mechanizmus vzniku trajektórie spravidla transparentnejší.

Záver

V tejto práci bola vykonaná podrobná analýza kinematiky robotických systémov, klasických algoritmov používaných na plánovanie pohybu v týchto systémoch, ako aj neurónových sietí, ich vlastností, typov a možností využitia v robotike. Zároveň sme sa oboznámili s učením s posilnením, jeho princípmi, oblasťami využitia a rôznymi typmi algoritmov.

Praktická časť bola venovaná robotu Panda [33] a simulátoru PyBullet, v rámci ktorých boli otestované štyri algoritmy založené na neurónových sieťach a následne porovnané s klasickými prístupmi. Výsledky praktickej časti ukázali, že neurónové siete sú v niektorých parametroch skutočne schopné prekonať tradičné algoritmy. Zároveň je však potrebné zdôrazniť, že tieto prístupy si stále vyžadujú ďalší vývoj a zlepšovanie. Hoci sú optimalizačné algoritmy spravidla pomalšie, vyznačujú sa vysokou plynulosťou, ktorá je v mnohých úlohách nevyhnutná. Na druhej strane sú neurónové siete adaptívnejšie a ich pretrénovanie si zvyčajne vyžaduje menej času ako príprava nového riešenia pri optimalizačných algoritmoch. Keďže boli realizované iba simulácie, viaceré algoritmy môžu byť v reálnych podmienkach náročné na implementáciu, najmä z dôvodu dynamických parametrov systému.

Do budúcnosti sa ponúka viacero možností ďalšieho rozvoja. Možné je otestovať väčší počet robotov s odlišným počtom DOF, pričom samotný simulátor podporuje aj iné roboty, ak je k dispozícii ich URDF model. Ich programovanie je však náročnejšie, keďže pre ne nie je pripravený taký hotový framework [predpripravená softvérová štruktúra] ako pre robota Panda. Ďalším smerom výskumu môže byť testovanie väčšieho počtu situácií a typov prostredí, prípadne aj dynamických scenárov. Podobné experimenty by bolo možné realizovať aj v prostredí ROS, kde by síce programovanie prebiehalo v odlišnej architektúre, no zároveň by mohlo byť štruktúrovanejšie a stabilnejšie.

Literatúra

1. SICILIANO, B.; KHATIB, O. *Springer Handbook of Robotics*. 2nd. Springer, 2016. Dostupné tiež z: <https://www.springer.com/gp/book/9783319325507>.
2. *IEEE Standard Ontologies for Robotics and Automation*. IEEE Standards Association, 2015. Č. IEEE Std 1872-2015. Dostupné tiež z: <https://ieeexplore.ieee.org/document/7370742>.
3. CRAIG, J. J. *Introduction to Robotics*. 3rd. 2005. Dostupné tiež z: <https://www.pearson.com/us/higher-education/program/Craig-Introduction-to-Robotics-3rd-Edition/PGM310589.html>.
4. CHOSET, H.; LYNCH, K. M.; HUTCHINSON, S.; KANTOR, G. A.; BURGARD, W. *Principles of robot motion: theory, algorithms, and implementations*. MIT press, 2005. Dostupné tiež z: <https://mitpress.mit.edu/books/principles-robot-motion>.
5. LIU, Y. et al. Creating better collision-free trajectory for robot motion planning by linearly constrained quadratic programming. *frontiers in Neurorobotics*. 2021, **15**, 724116. Dostupné tiež z: <https://www.frontiersin.org/articles/10.3389/fnbot.2021.724116/full>.
6. GASPARETTO, A.; BOSCARIOL, P.; LANZUTTI, A.; VIDONI, R. Path Planning and Trajectory Planning Algorithms: A General Overview. In: *Mechanisms and Machine Science*. Springer, 2015. Dostupné z DOI: 10.1007/978-3-319-14705-5_1. Figure “C-space, C-free and C-obs for an articulated robot with two joints”.
7. LAVALLE, S. M. *Planning algorithms*. Cambridge university press, 2006. Dostupné tiež z: <https://planning.cs.uiuc.edu/>.
8. YANG, Y.; PAN, J.; WAN, W. Survey of optimal motion planning. *IET Cyber-systems and Robotics*. 2019, **1**(1), 13–19. Dostupné tiež z: <https://onlinelibrary.wiley.com/doi/full/10.1049/iet-csr.2018.0003>.
9. DOBIŠ, M.; DEKAN, M.; BEŇO, P.; DUCHOŇ, F.; BABINEC, A. Evaluation criteria for trajectories of robotic arms. *Robotics*. 2022, **11**(1), 29. Dostupné tiež z: <https://www.mdpi.com/2218-6581/11/1/29>.
10. EVEN, S. *Graph algorithms*. Cambridge University Press, 2011.

11. NASH, A.; DANIEL, K.; KOENIG, S.; FELNER, A. Theta*: Any-Angle Path Planning on Grids. In: *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*. 2007, s. 1177–1183. Dostupné tiež z: <https://aaai.org/papers/01177-aaai07-187-theta-any-angle-path-planning-on-grids/>.
12. KOENIG, S.; LIKHACHEV, M. D* Lite. In: *Proceedings of the Eighteenth National Conference on Artificial Intelligence*. 2002, s. 476–483. Dostupné tiež z: <https://www.ri.cmu.edu/publications/d-lite/>.
13. RATLIFF, N.; ZUCKER, M.; BAGNELL, J. A.; SRINIVASA, S. CHOMP: Gradient Optimization Techniques for Efficient Motion Planning. In: *Proceedings of the 2009 IEEE International Conference on Robotics and Automation*. 2009, s. 489–494. Dostupné tiež z: <https://publications.ri.cmu.edu/chomp-gradient-optimization-techniques-for-efficient-motion-planning>.
14. KALAKRISHNAN, M.; CHITTA, S.; THEODOROU, E.; PASTOR, P.; SCHAAAL, S. STOMP: Stochastic Trajectory Optimization for Motion Planning. In: *Proceedings of the 2011 IEEE International Conference on Robotics and Automation*. 2011, s. 4569–4574. Dostupné tiež z: https://is.mpg.de/am/publications/kalakrishnan_raiic_2011.
15. SCHULMAN, J. et al. Motion Planning with Sequential Convex Optimization and Convex Collision Checking. *The International Journal of Robotics Research*. 2014, **33**(9), 1251–1270. Dostupné tiež z: <https://rll.berkeley.edu/~sachin/papers/Schulman-IJRR2014.pdf>.
16. KHATIB, O. Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. *The International Journal of Robotics Research*. 1986, **5**(1), 90–98. Dostupné tiež z: https://khatib.stanford.edu/publications/pdfs/Khatib_1986_IJRR.pdf.
17. KAVRAKI, L. E.; ŠVESTKA, P.; LATOMBE, J.-C.; OVERMARS, M. H. Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces. *IEEE Transactions on Robotics and Automation*. 1996, **12**(4), 566–580. Dostupné tiež z: https://web.ics.purdue.edu/~rvoyles/Courses/ROSprogramming/Kavraki_Latombe.PRM_PathPlanning.TRA96.pdf.
18. KUFFNER, J. J.; LAVALLE, S. M. RRT-Connect: An Efficient Approach to Single-Query Path Planning. In: *Proceedings of the 2000 IEEE International Conference on Robotics and Automation*. 2000, s. 995–1001. Dostupné tiež z: https://www.researchgate.net/publication/221075120_RRT-Connect_An_Efficient_Approach_to_Single-Query_Path_Planning.

19. LAVALLE, S. M. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. 1998. Tech. spr., TR 98-11. Iowa State University. Dostupné tiež z: <https://lavalle.pl/papers/Lav98c.pdf>.
20. GURNEY, K. *An introduction to neural networks*. CRC press, 2018. Dostupné tiež z: <https://www.taylorfrancis.com/books/mono/10.1201/9781315273570/introduction-neural-networks-kevin-gurney>.
21. KANDEL, E. R. et al. *Principles of neural science*. Zv. 4. McGraw-hill New York, 2000. Dostupné tiež z: <https://accessmedicine.mhmedical.com/book.aspx?bookID=1049>.
22. OTOMAR, K. et al. *Lékařská fyziologie*. Grada Publishing as, 2011. Dostupné tiež z: https://books.google.sk/books?hl=ru&lr=&id=kkqECwAAQBAJ&oi=fnd&pg=PA1&dq=Otomar,+Kittnar++fyziologie&ots=60vzxrA9Vx&sig=5Z3-_Eqkq6QyGFFzrfvIeq97BvY&redir_esc=y#v=onepage&q=Otomar%2C%20Kittnar%20%20fyziologie&f=false.
23. NAIR, V.; HINTON, G. E. Rectified Linear Units Improve Restricted Boltzmann Machines. *Proceedings of the 27th International Conference on Machine Learning*. 2010, 807–814. Dostupné tiež z: <https://www.cs.toronto.edu/~hinton/absps/reluICML.pdf>.
24. O'SHEA, K.; NASH, R. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*. 2015. Dostupné tiež z: <https://arxiv.org/abs/1511.08458>.
25. SUTTON, R. S.; BARTO, A. G. et al. *Reinforcement learning: An introduction*. Zv. 1. MIT press Cambridge, 1998. Č. 1. Dostupné tiež z: <http://incompleteideas.net/book/the-book-2nd.html>.
26. OPENAI. *Spinning Up in Deep RL*. 2018. Dostupné tiež z: <https://spinningup.openai.com/en/latest/index.html>.
27. SCHULMAN, J.; WOLSKI, F.; DHARIWAL, P.; RADFORD, A.; KLIMOV, O. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*. 2017. Dostupné tiež z: <https://arxiv.org/abs/1707.06347>.
28. HAARNOJA, T.; ZHOU, A.; ABBEEL, P.; LEVINE, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In: *Proceedings of the 35th International Conference on Machine Learning*. 2018, s. 1861–1870. Dostupné tiež z: <https://proceedings.mlr.press/v80/haarnoja18b.html>.

29. FUJIMOTO, S.; HOOFF, H. van; MEGER, D. Addressing Function Approximation Error in Actor-Critic Methods. In: *Proceedings of the 35th International Conference on Machine Learning*. 2018, s. 1587–1596. Dostupné tiež z: <https://proceedings.mlr.press/v80/fujimoto18a.html>.
30. KUZNETSOV, A.; SHVECHIKOV, P.; GRISHIN, A.; VETROV, D. Controlling Overestimation Bias with Truncated Mixture of Continuous Distributional Quantile Critics. In: *Proceedings of the 37th International Conference on Machine Learning*. 2020, s. 5556–5566. Dostupné tiež z: <https://proceedings.mlr.press/v119/kuznetsov20a.html>.
31. RAFFIN, A. et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*. 2021, **22**(268), 1–8. Dostupné tiež z: <http://jmlr.org/papers/v22/20-1364.html>.
32. TOWERS, M. et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. 2024. Dostupné z arXiv: 2407.17032 [cs.LG].
33. GALLOUÉDEC, Q.; CAZIN, N.; DELLANDRÉA, E.; CHEN, L. panda-gym: Open-Source Goal-Conditioned Environments for Robotic Learning. *4th Robot Learning Workshop: Self-Supervised and Lifelong Learning at NeurIPS*. 2021. Dostupné tiež z: <https://panda-gym.readthedocs.io/en/latest/index.html>.
34. COUMANS, E.; BAI, Y.; HSU, J. *pybullet: Official Python Interface for the Bullet Physics SDK specialized for Robotics Simulation and Reinforcement Learning* [<https://pypi.org/project/pybullet/>]. 2025. Version 3.2.7; released Jan 30, 2025; accessed: 2026-04-29.
35. FRANKA ROBOTICS. *Robot and Interface Specifications* [https://support.franka.de/docs/control_parameters.html]. 2024. Franka Control Interface documentation; accessed: 2026-05-26.
36. FRANKA ROBOTICS. *libfranka* [<https://support.franka.de/docs/libfranka.html>]. 2024. Franka Control Interface documentation; accessed: 2026-05-26.
37. LOBBEZOO, A.; KWON, H.-J. Simulated and Real Robotic Reach, Grasp, and Pick-and-Place Using Combined Reinforcement Learning and Traditional Controls. *Robotics*. 2023, **12**(1), 12. Dostupné z DOI: 10.3390/robotics12010012.
38. ZHAO, W.; QUERALTA, J. P.; WESTERLUND, T. Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey. In: *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*. 2020, s. 737–744. Dostupné z DOI: 10.1109/SSCI47803.2020.9308555.

39. RIZZARDO, C.; CHEN, F.; CALDWELL, D. G. Sim-to-real via latent prediction: Transferring visual non-prehensile manipulation policies. *Frontiers in Robotics and AI*. 2023, **9**, 1067502. Dostupné z DOI: 10.3389/frobt.2022.1067502.

Použitie nástrojov umelej inteligencie

Čestne vyhlasujem, že bakalárska práca s názvom **Aplikácia umelej inteligencie pri plánovaní pohybu robotických systémov** bola vypracovaná samostatne na základe zadania práce a s odbornou pomocou vedúceho práce, ako aj z verejne dostupných zdrojov.

Pri vykonávaní práce bola umelá inteligencia [ChatGPT 5.0-5.5] využitá na jazykovú korekciu, tiež ako konzultačná podpora v rámci praktickej časti úlohy a na implementáciu algoritmov. Umelá inteligencia **nebola** použitá na generovanie štatistických údajov z experimentov; všetky výstupy a výsledky boli overené autorom na základe odbornej literatúry.

Dodatok A: Definícia úlohy

Algoritmus 1 Trieda úlohy Reach

```
class PandaReachWallTask(Reach): # Definícia vlastnej úlohy
    #inicializácia úlohy, vytvorenie steny v simulácii
    def __init__(self, sim, get_ee_position, reward_type="dense"):
        super().__init__(sim, #konštruktor nadtriedy Reach #dedictvo
            get_ee_position=get_ee_position, #poloha EE
            reward_type=reward_type, distance_threshold=0.01) #blízkosť k cieľu 1cm
        self.wall_x = 0.20 #poloha steny v osi X
        self.wall_width = 0.01 #hrúbka steny
        self.wall_height = 0.2 #výška steny

    # Vytvorenie kvádra reprezentujúceho prekážku v simulácii
    self.sim.create_box(
        body_name="wall", #Názov objektu
        #Polovičné rozmery kvádra (šírka, hĺbka, výška)
        half_extents=[self.wall_width/2, 0.2, self.wall_height],
        mass=0, # Nulová hmotnosť (znamená, že stena je statická)
        #Poloha stredu steny v karteziánskom priestore
        position=[self.wall_x, 0.0, self.wall_height/2],
        rgba_color=[0.8, 0.1, 0.1, 1.0] # Farba steny vo formáte RGBA
    )

    def reset(self): # Obnovenie úlohy na začiatku novej epizódy
        # Nastavenie cieľovej polohy EE
        self.goal = np.array([0.4, 0.0, 0.1])
        # Presunutie vizuálneho cieľa na nastavenú polohu bez rotácie
        self.sim.set_base_pose("target", self.goal, np.array([0, 0, 0, 1]))
```

Dodatok B: Nastavenia príslušných algoritmov

Táto príloha uvádza nastavenia algoritmov použitých v experimentálnej časti práce. Hodnoty boli prevzaté zo zdrojových súborov v priečinku `prax`. Pri algoritmoch učenia s posilňovaním ide o predvolené hodnoty v tréningových skriptoch. Ak sa skript spustil s vlastnými argumentmi príkazového riadka, skutočná hodnota sa mohla prepísať.

Spoločné nastavenie prostredia

Vo všetkých troch prípadoch bolo použité robotické rameno Franka Emika Panda s riadením v kĺbovom priestore. Počiatočná konfigurácia kĺbov bola nastavená na

$$q_0 = [2.7, 0.2, 0.2, -1.0, 0.0, 1.7, 0.7].$$

Pre všetky prípady bola cieľová poloha koncového efektora

$$p_g = [0.4, 0.0, 0.1],$$

Predvolený typ odmeny v tréningových skriptoch je `dense` a predvolená maximálna dĺžka epizódy je 300 krokov.

Nastavenia neurónových sietí (SAC, TD3, PPO, TQC)

Argument `-total-timesteps` má predvolenú hodnotu 150 000. Vyhodnocovanie prebieha každých 10 000 krokov a používa 5 epizód.

Všeobecne platilo, že všetky siete mali LR(learning rate, rýchlosť učenia - veľkosť kroku pri aktualizácii váh siete) = 0.001, `batch_size`(počet vzoriek spracovaných v jednej tréningovej dávke) = 256 a `buffer_size` = 300 000. Argument `seed` má predvolenú hodnotu `None`, **pri spustení sa však použilo náhodné kladné celé číslo ako argument príkazového riadka.**

Podrobné všeobecné a špecifické parametre, ktoré platia vo všetkých prípadoch, sú uvedené v tabuľke nižšie:

Algoritmus	Hyperparametre
PPO	<code>policy = MultiInputPolicy; n_steps = 2048; gamma = 0.99; gae_lambda = 0.95; clip_range = 0.2; ent_coef = 0.0; vf_coef = 0.5; max_grad_norm = 0.5; verbose = 1.</code>
SAC	<code>policy = MultiInputPolicy; learning_starts = 5000; tau = 0.005; gamma = 0.99; train_freq = 1; gradient_steps = 1; verbose = 1.</code>
TD3	<code>policy = MultiInputPolicy; learning_starts = 5000; tau = 0.005; gamma = 0.99; policy_delay = 2; action_noise = NormalActionNoise; sigma = action_noise_std = 0.15; verbose = 1.</code>
TQC	<code>policy = MultiInputPolicy; learning_starts = 5000; tau = 0.005; gamma = 0.99; train_freq = 1; gradient_steps = 1; top_quantiles_to_drop_per_net = 2; verbose = 1.</code>

Legenda k hyperparametrom:

policy určuje typ politiky použitej modelom;

n_steps určuje počet krokov interakcie s prostredím pred jednou aktualizáciou modelu;

gamma je diskontný faktor určujúci význam budúcich odmien;

gae_lambda je parameter metódy Generalized Advantage Estimation a ovplyvňuje kompromis medzi odchýlkou a rozptylom odhadu výhody;

clip_range určuje rozsah orezania zmien politiky v algoritme PPO;

ent_coef predstavuje váhu entropickej zložky podporujúcej exploráciu;

vf_coef určuje váhu straty hodnotovej funkcie v celkovej stratovej funkcii;

max_grad_norm nastavuje maximálnu normu gradientu pri jeho orezávaní;

verbose určuje úroveň textového výstupu počas tréningu;

buffer_size označuje veľkosť pamäťového zásobníka skúseností;

learning_starts určuje počet krokov nazbieraných pred začiatkom učenia;

tau predstavuje koeficient mäkkej aktualizácie cieľových sietí;

train_freq určuje frekvenciu tréningových aktualizácií vzhľadom na interakcie s prostredím;

gradient_steps označuje počet optimalizačných krokov vykonaných po jednej tréningovej aktivácii;

policy_delay určuje oneskorenie aktualizácie politiky voči aktualizácii kritika v algoritme TD3;

action_noise predstavuje šum pridávaný k akciám s cieľom podporiť exploráciu;

sigma určuje intenzitu tohto šumu;

top_quantiles_to_drop_per_net vyjadruje počet najvyšších kvantilov ignorovaných pri učení v algoritme TQC s cieľom obmedziť nadhodnocovanie akčných hodnôt.

Konfigurácia klasických algoritmov

Klasické algoritmy nemajú samostatné konfiguračné súbory. Ich nastavenia sú uložené priamo v príslušných spúšťacích skriptoch. Tabuľka preto uvádza súbor, v ktorom je uložená konfigurácia použitého algoritmu.

Prípad	Algoritmus	Zdrojový súbor s konfiguráciou
0	PRM	Vybrané hodnoty: <code>num_samples = 220</code> , <code>k_neighbors = 12</code> , <code>edge_substeps = 16</code> , <code>q_min/q_max = [-3.0, 3.0]</code> , <code>log_dt = 0.02</code> .
0	CHOMP	Vybrané hodnoty: <code>num_timesteps = 26</code> , <code>num_iterations = 120</code> , <code>step_size = 0.12</code> , <code>smoothness_weight = 1.0</code> , <code>regularization_weight = 1e-3</code> , <code>initial_bend = 0.18</code> .
1	PRM	Vybrané hodnoty: <code>num_samples = 200</code> , <code>k_neighbors = 10</code> , <code>max_steps_per_waypoint = 50</code> , <code>waypoint_tolerance = 0.02</code> , <code>log_dt = 0.02</code> .
1	STOMP	Vybrané hodnoty: <code>num_timesteps = 26</code> , <code>num_iterations = 60</code> , <code>num_samples = 24</code> , <code>noise_std = 0.05</code> , <code>obstacle_weight = 18.0</code> , <code>safety_margin = 0.06</code> , <code>update_rate = 0.35</code> .
2	RRT-Connect	Vybrané hodnoty: <code>max_iterations = 2500</code> , <code>max_step = 0.24</code> , <code>edge_substeps = 20</code> , <code>goal_bias = 0.12</code> , <code>safety_margin = 0.012</code> , <code>log_dt = 0.02</code> .
2	TrajOpt	Vybrané hodnoty: <code>num_timesteps = 32</code> , <code>num_iterations = 120</code> , <code>trust_region = 0.16</code> , <code>smoothness_weight = 1.0</code> , <code>obstacle_weight = 24.0</code> , <code>goal_weight = 8.0</code> , <code>safety_margin = 0.06</code> .

Legenda k parametrom:

num_samples určuje počet vzoriek generovaných pri plánovaní;
k_neighbors je počet susedných uzlov zohľadnených pri vytváraní cestnej mapy;
edge_substeps je počet medzikrokov použitých pri kontrole priechodnosti úseku trajektórie;
log_dt predstavuje časový krok použitý pri zázname alebo vyhodnotení trajektórie;
q_min/q_max určujú minimálne a maximálne limity kľbových premenných;
num_timesteps označuje počet časových krokov trajektórie;
num_iterations vyjadruje počet optimalizačných iterácií;
step_size predstavuje veľkosť kroku pri optimalizácii;
smoothness_weight určuje váhu kritéria hladkosti trajektórie;
regularization_weight su váhy regularizačného člena stabilizujúceho optimalizáciu;
initial_bend určuje počiatkové zakrivenie alebo odklon počiatkovej trajektórie;
max_steps_per_waypoint je maximálny počet krokov medzi dvoma bodmi trajektórie;
waypoint_tolerance určuje toleranciu dosiahnutia bodu trajektórie;
noise_std je smerodajnou odchýlkou šumu použitého pri generovaní perturbácií;
obstacle_weight určuje váhu penalizácie za blízkosť alebo kolíziu s prekážkami;
safety_margin predstavuje bezpečnostnú rezervu od prekážok;
update_rate vyjadruje mieru, akou sa trajektória aktualizuje medzi iteráciami;
max_iterations určuje maximálny počet iterácií plánovacieho algoritmu;
max_step označuje maximálnu dĺžku jedného kroku pri rozširovaní stromu;
goal_bias vyjadruje pravdepodobnosť preferovania cieľa pri vzorkovaní;
trust_region určuje povolený rozsah zmeny riešenia v jednej optimalizačnej iterácii;
goal_weight predstavuje váhu kritéria smerujúceho trajektóriu k cieľovému stavu.